

A NEW INPUT GROUPING AND SHARING METHOD TO DESIGN LOW COMPLEXITY 32 – POINT FFT IMPLEMENTATION

A Project Report

Submitted to the FACULTY of ENGINEERING of

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA

In partial fulfillment of the requirements,

For the award of the Degree of

Bachelor of Technology

In

Electronics and Communication Engineering

By

Venu Vaddimukkala

(22481A04N9)

Saniya Begum

(22481A04L3)

Yasam Gopalakrishna

(22481A04P2)

Rokkam Varshitha

(22481A04L1)

Under the Guidance of

Dr. V. V. K. D. V. Prasad

Professor of ECE & Convenor-IQAC, SRGEC.



Department of Electronics and Communication Engineering

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE

GUDLAVALLERU - 521356

ANDHRA PRADESH

2025-2026

A NEW INPUT GROUPING AND SHARING METHOD TO DESIGN LOW COMPLEXITY 32 – POINT FFT IMPLEMENTATION

A Project Report

Submitted to the FACULTY of ENGINEERING of

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA

In partial fulfillment of the requirements,

For the award of the Degree of

Bachelor of Technology

In

Electronics and Communication Engineering

By

Venu Vaddimukkala
(22481A04N9)

Saniya Begum
(22481A04L3)

Yasam Gopalakrishna
(22481A04P2)

Rokkam Varshitha
(22481A04L1)

Under the Guidance of

Dr. V. V. K. D. V. Prasad

Professor of ECE & Convenor-IQAC, SRGEC.



Department of Electronics and Communication Engineering

SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE

(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)

SESHADRI RAO KNOWLEDGE VILLAGE

GUDLAVALLERU - 521356

ANDHRA PRADESH

2025-2026

Department of Electronics and Communication Engineering
SESHADRI RAO GUDLAVALLERU ENGINEERING COLLEGE
(An Autonomous Institute with Permanent Affiliation to JNTUK, Kakinada)
SESHADRI RAO KNOWLEDGE VILLAGE
GUDLAVALLERU - 521356



CERTIFICATE

This is to certify that the project report entitled “**A New Input Grouping and Sharing Method to Design Low Complexity 32 – Point FFT Implementation**” is a Bonafide record of work carried out by **Vaddimukkala Venu (22481A04N9)**, **Saniya Begum (22481A04L3)**, **Yasam Gopalakrishna (22481A04P2)**, **Rokkam Varshitha (22481A04L1)** under my guidance and supervision in partial fulfillment of the requirements, for the award of the degree of Bachelor of Technology in **Electronics and Communication Engineering** of Seshadri Rao Gudlavalleru Engineering affiliated to **Jawaharlal Nehru Technological University, Kakinada**.

(Dr. V.V.K.D.V. PRASAD)
Project Supervisor

(Dr. B. RAJASEKHAR)
Head of the Department

Acknowledgement

We are very glad to express our deep sense of gratitude to **Dr. V. V. K. D. V. Prasad**, Professor, Electronics and Communication Engineering for their guidance and cooperation in completing this project. We convey our heartfelt thanks to them for their inspiring assistance till the end of our project.

We convey our sincere and indebted thanks to our beloved Head of the Department **Dr. B. Rajasekhar**, for his encouragement and help for completing our project successfully.

We also extend our gratitude to our Principal **Dr. B. Karuna Kumar**, for the support and for providing facilities required for the completion of our project.

We impart our heartfelt gratitude to all the Lab Technicians for helping us in all aspects related to our project.

We thank our friends and all others who rendered their help directly and indirectly to complete our project.

Vaddimukkala Venu (22481A04N9)

Saniya Begum (22481A04L3)

Yasam Gopalakrishna (22481A04P2)

Rokkam Varshitha (22481A04L1)

CONTENTS

TITLE	PAGE NO
LIST OF FIGURES	i
LIST OF TABLES	ii
ABSTRACT	iii
1. Introduction	
1.1 Background	1
1.2 Aim of this Project	1
1.3 Methodology	1
1.4 Significance of this Work	2
1.5 Outline	2
2. Literature Review	3
3. FFT Implementation	7
3.1 Exiting Method	7
3.1.1 Input Data	9
3.1.2 Input Grouping	9
3.1.3 Partial Sum Sharing	9
3.1.4 UAS Units in The Architecture	9
3.1.5 Butterfly Structure	10
3.1.6 Stages of the FFT	10
3.1.7 Real, Imaginary, and Complex Outputs	10
3.1.8 Twiddle Factors	10
3.1.9 Data Flow and Connectivity	10
3.1.10 Processing of Even and Odd Data Points	10
3.1.11 Recursive Computation in FFT	10
3.1.12 Optimization through In-Place Computation	11
3.1.13 Handling of Complex Numbers	11
3.1.14 Importance of the FFT in Signal Processing	11
3.1.15 Parallelization in FFT Computation	11
3.1.16 Handling of Input and Output Data in FFT	11
3.1.17 Efficiency of FFT Over Direct DFT	11
3.1.18 Twiddle Factor Optimization	11
3.1.19 Memory Consideration FFT Implementation	11
3.1.20 Hardware Implementation of FFT	12

4. Proposed Architecture and Method	13
4.1 Proposed 32-Point Architecture	13
4.2 Proposed Method	13
4.2.1 Flip flop Based Clock Gating Technique	13
4.3 Implementation of Proposed Architecture using Verilog HDL	15
4.4 RTL Schematic for Proposed Architecture	15
4.5 Technology Schematic for Proposed Architecture	16
5. Xilinx Vivado Software	18
5.1 Benefits	18
5.2 Features	19
5.3 Vivado Design Flow	19
6. Results	24
6.1 32-Point FFT Without Clock Gating	24
6.1.1 Simulation Waveform	24
6.1.2 Area Utilization	24
6.1.3 Delay Analysis	25
6.1.4 Power Consumption	25
6.2 32 – point FFT With Clock Gating	26
6.2.1 Simulation Waveform	26
6.2.2 Area Utilization	26
6.2.3 Delay Analysis	27
6.2.4 Power Consumption	27
6.3 Results Comparison	28
6.4 Applications	28
6.5 Advantages	29
7. Conclusion and Future Scope	31
REFERENCES	32
PROJECT COs– POs/PSOs MAPPING	35
APENDIX	38

LIST OF FIGURES

Fig. No.	NAME OF THE FIGURE	Page No.
3.1	The Existing 16 – Point FFT Architecture	8
3.2	UAS Architecture	9
4.1	Flip flop-based Clock gating technique	13
4.2	The Proposed 32 – Point FFT Architecture	14
4.3	RTL Schematic of Proposed Architecture	16
4.4	Technology Schematic of Proposed Architecture	17
5.1	New Project Wizard – Project Type Page	20
5.2	Implemented design View in Vivado Design Suite	21
5.3	Project Setting Window	22
6.1	32-Point FFT Architecture Without Clock Gating Simulation Waveform	24
6.2	Device Utilization	24
6.3	Timing Analysis Report	25
6.4	Power Analysis Report	25
6.5	32 – Point FFT With Clock Gating Simulation Waveform	26
6.6	Device Utilization	26
6.7	Timing Analysis Report	27
6.8	Power Analysis Report	27

LIST OF TABLES

Table No.	Name of the Table	Page no.
4.1	Clock Gating Truth Table	14
6.1	Comparison of Results	28

ABSTRACT

The Fast Fourier Transform (FFT) is a fundamental algorithm in signal processing, widely used for efficient computation of frequency-domain representations of signals. However, optimizing FFT for low complexity, especially for resource-constrained environments, remains a challenging task. This paper introduces an innovative input grouping and sharing approach for the design of a low-complexity 32-point FFT implementation. The proposed method focuses on reducing the computational burden by strategically grouping input data and sharing intermediate results, thereby minimizing the number of required operations. This technique leverages the symmetry and periodicity inherent in FFT algorithms, ensuring a more efficient use of hardware resources. Through detailed analysis and comparisons with traditional FFT designs, the paper demonstrates that the proposed approach significantly reduces both computational complexity and power consumption while maintaining high accuracy and performance. The results highlight the potential of this technique for applications in embedded systems, mobile devices, and real-time signal processing tasks, where both performance and efficiency are crucial.

Keywords— *FFT, twiddle factors, input grouping, partial sum sharing.*

Chapter 1

INTRODUCTION

1.1 Background

The Fast Fourier Transform (FFT) is a fundamental algorithm in digital signal processing and is extensively used in applications such as communication systems, image processing, spectral analysis, radar systems, and biomedical signal processing. FFT enables efficient computation of the Discrete Fourier Transform (DFT) by reducing computational complexity, thereby making real-time signal processing feasible. However, despite its advantages, the implementation of FFT in hardware presents several challenges, particularly in terms of high computational complexity, increased power consumption, and large silicon area. These challenges mainly arise due to the extensive use of multiplications and additions involving twiddle factor coefficients during butterfly operations. In conventional FFT architectures, redundant arithmetic operations lead to inefficient utilization of hardware resources. With the increasing demand for low-power and high-performance systems in embedded platforms, IoT devices, and wireless communication systems, there is a strong requirement for FFT architectures that offer reduced complexity and improved energy efficiency. To address these issues, optimization techniques such as input grouping and partial sum sharing have been introduced to minimize redundant computations while maintaining accuracy and performance.

1.2 Aim of this project

The aim of this project is to design and analyze a low-complexity Fast Fourier Transform architecture using an innovative input grouping and partial sum sharing approach. The primary objective is to reduce the number of arithmetic operations, particularly multiplications involving twiddle factors, thereby minimizing hardware complexity, power consumption, and area. The project focuses on the implementation and evaluation of a 32-point FFT architecture suitable for real-time digital signal processing applications.

1.3 Methodology

The methodology adopted in this project involves the design of an optimized FFT architecture by identifying and eliminating redundant arithmetic operations present in conventional FFT implementations. Input samples are grouped strategically, and intermediate partial sums are shared across multiple stages of computation to reduce repeated calculations. This approach significantly decreases the number of multipliers and adders required in the architecture. The proposed FFT design is implemented using hardware description techniques and evaluated through simulation and logic synthesis. Performance parameters such as area, power consumption, and delay are analysed and compared with existing FFT architectures to validate the effectiveness of the proposed method.

1.4 Significance of this work

The proposed FFT architecture offers significant improvements in terms of reduced computational complexity and lower power consumption, making it suitable for modern digital signal processing systems. By efficiently utilizing hardware resources and minimizing redundant operations, the design achieves improved energy efficiency and reduced silicon area. This work is particularly important for applications in wireless communication systems, embedded devices, IoT platforms, image and audio processing, and real-time signal analysis. The scalable nature of the proposed approach allows it to be extended to higher-order FFT sizes, thereby enhancing its applicability in future high-performance and low-power DSP applications.

1.5 Outline

Chapter 1: It deals with the background and outline of the project and the base idea regarding the project.

Chapter 2: It deals with the literature survey existing system and discusses about the efficiency of the existing method.

Chapter 3: It deals with the work title explanation and block diagram explanation.

Chapter 4: It deals with the implementation of the project and their required functioning to operate the necessary parts of the prototype.

Chapter 5: It deals with the software tools required and their usability and contribution to the proper functioning of the project.

Chapter 6: It deals with the result of the project. Advantages and disadvantages, efficiency of the project are discussed.

Chapter 7: It deals with the conclusion the project.

Chapter 2

LITERATURE REVIEW

W. Hong et al., “Multibeam antenna technologies for 5G wireless communications,” *IEEE Trans. Antennas Propag.*, vol. 65, no. 12, pp. 6231–6249, Dec. 2017 [1] Multibeam antenna technologies play a critical role in enhancing the performance and efficiency of 5G wireless communication systems by enabling simultaneous communication with multiple users or devices. These technologies utilize advanced beamforming techniques to generate and steer multiple narrow beams, significantly improving spectral efficiency, data rates, and coverage. By focusing energy in specific directions, multibeam antennas reduce interference and enhance signal quality, making them ideal for high-density urban environments and massive MIMO (Multiple Input Multiple Output) deployments. They support dynamic user allocation, enabling seamless communication and improved network capacity while addressing the diverse requirements of 5G services, such as ultra-reliable low-latency communication (URLLC) and enhanced mobile broadband (eMBB). Furthermore, multibeam antennas optimize power usage and spatial resources, making them essential for achieving the high-speed, low-latency, and high-capacity goals of 5G networks. Their integration into base stations and small cells allows for efficient spectrum utilization and supports the increasing demand for data traffic in next-generation wireless systems.

P. K. Daram and C.-I. H. Chen, “Smart FFT measurement for reconfigurable sensor using a wideband digital receiver,” in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, Aug. 2021, pp. 200–203 [2] The paper titled "Smart FFT Measurement for Reconfigurable Sensor Using a Wideband Digital Receiver" presents an efficient approach to implement a Fast Fourier Transform (FFT)-based measurement system tailored for reconfigurable sensors operating within wideband digital receivers. The proposed system leverages the adaptability of reconfigurable sensors to dynamically analyze signals across a broad frequency spectrum while optimizing performance and hardware resource utilization. By integrating smart FFT techniques, the design enables precise spectral measurements, ensuring enhanced signal detection and processing capabilities. The wideband digital receiver facilitates high-speed data acquisition, while the reconfigurable architecture ensures flexibility to accommodate varying signal conditions and system requirements. This combination makes the proposed system suitable for modern applications requiring robust, low-power, and scalable solutions for real-time signal monitoring and analysis across multiple frequency bands.

J. Chen, J. Hu, S. Lee, and G. E. Sobelman, “Hardware efficient mixed radix-25/16/9 FFT for LTE systems,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 2, pp. 221–229, Feb. 2015 [3] The hardware-efficient mixed radix-25/16/9 FFT is designed to meet the computational demands of LTE systems by optimizing the balance between performance, power, and resource usage. This FFT architecture combines multiple radix sizes—25, 16, and 9—to efficiently process varying FFT lengths (e.g., 128, 256, 512, and 1024 points) used in LTE downlink and uplink channels. By leveraging the mixed-radix approach, the design reduces arithmetic complexity, minimizes the number of multipliers, and improves data flow through optimized butterfly operations. The radix-25 stage handles larger factors, reducing the depth of computation, while radix-16 and radix-9 stages allow finer granularity to address smaller factors efficiently. This

hierarchical structure reduces the overall computational load, power consumption, and hardware cost compared to conventional FFT designs. The proposed architecture is particularly well-suited for real-time LTE applications, providing high throughput with minimal latency, making it ideal for implementation on ASICs or FPGAs for next-generation communication systems.

N. Lei and X. Gu, “FFT-OT: A fast algorithm for optimal transportation,” in Proc. IEEE/CVF ICCV, Oct. 2021, pp. 6260–6269 [4] FFT-OT is a fast algorithm designed to solve the Optimal Transportation (OT) problem efficiently by leveraging the Fast Fourier Transform (FFT). Optimal Transportation, which involves finding the most cost-efficient way to transform one probability distribution into another, is computationally intensive due to its reliance on pairwise distance computations. FFT-OT addresses this challenge by accelerating the computation of the convolution-like operations inherent in regularized OT formulations, such as those using entropic regularization. By representing the problem in the frequency domain through FFT, the algorithm significantly reduces the computational complexity compared to traditional OT solvers, making it more scalable for large-scale data. This approach retains accuracy while achieving faster runtimes, making FFT-OT highly suitable for applications in machine learning, image processing, and data science where OT plays a critical role in tasks like distribution alignment, domain adaptation, and generative modeling.

W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” Math. Comput., vol. 19, pp. 297–301, Apr. 1965 [5] The algorithm for the machine calculation of complex Fourier series, commonly referred to as the Fast Fourier Transform (FFT), was introduced by James Cooley and John Tukey in 1965. It revolutionized the computation of the Discrete Fourier Transform (DFT) by reducing its complexity from $O(N^2)$ to $O(N \log N)$, where N is the number of data points. This efficiency is achieved by recursively breaking down the DFT into smaller sub-transforms, exploiting the symmetry and periodicity properties of the complex exponential terms (twiddle factors). The algorithm efficiently rearranges input data using a "divide and conquer" strategy, often implemented with butterfly operations in a radix-2 structure for power-of-two input sizes. This approach significantly reduces computational time and hardware resource usage, enabling real-time signal processing and spectral analysis in applications such as telecommunications, audio, radar, and image processing. The FFT's versatility and efficiency have cemented its importance as a foundational tool in digital signal processing and beyond.

J. Wang, S. Li, and X. Li, “Scheduling of data access for the radix-2k FFT processor using single-port memory,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 28, no. 7, pp. 1676–1689, Jul. 2020. [6] The scheduling of data access for the radix-2^k FFT processor using single-port memory focuses on optimizing memory access patterns to efficiently handle the butterfly computations required in FFT operations. Since single-port memory allows only one read or write operation per clock cycle, careful data scheduling is essential to avoid conflicts and ensure continuous data flow. In radix-2^k FFT architectures, the data is divided into groups, and the FFT stages process these groups sequentially. By adopting a conflict-free memory addressing scheme, intermediate results are systematically stored and retrieved to minimize latency and maximize resource utilization. The data scheduling scheme often employs techniques such as in-place computation and address mapping to ensure that consecutive memory accesses align with the butterfly operation requirements. This approach effectively reduces memory complexity and

access delays while maintaining the throughput of the radix-2^k FFT processor, making it well-suited for hardware-constrained systems that rely on single-port memory.

J. Hazarika, M. T. Khan, and S. R. Ahamed, “Low-complexity continuous-flow memory-based FFT architectures for real-valued signals,” in Proc. 32nd Int. Conf. VLSI Des. 18th Int. Conf. Embedded Syst. (VLSID), Jan. 2019, pp. 46–51 [7] Low-complexity continuous-flow memory-based FFT architectures for real-valued signals focus on optimizing the processing of real-valued inputs, which are common in many digital signal processing applications. optimization, and resource sharing to minimize redundant operations, such as eliminating unnecessary multiplication and additions. By employing continuous-flow processing, where input data streams through the FFT pipeline without requiring large intermediate memory buffers, these designs achieve high throughput while keeping hardware overhead low. Memory-based approaches leverage compact memory structures to store intermediate results efficiently, further reducing area and power consumption. Overall, these architectures strike an optimal balance between computational efficiency, hardware simplicity, and throughput, making them ideal for real-time and energy-efficient applications in communication systems, audio processing, and embedded systems.

S. Liu and D. Liu, “A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 27, no. 3, pp. 511–523, Mar. 2019 [8] A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G applications is designed to efficiently handle the computational demands of fast Fourier transforms (FFT) in modern communication systems. The processor uses a memory-based architecture that enables flexibility in handling different FFT sizes and modulation schemes required by these wireless standards. By utilizing high-speed memory access and optimized data flow management, it minimizes latency and improves throughput, ensuring real-time signal processing for high-speed data transmission. This architecture is scalable, making it suitable for a range of applications, from 4G LTE to the evolving requirements of 5G, supporting wideband communications and massive MIMO systems. Additionally, it integrates hardware accelerators and pipelined processing to ensure low power consumption while maintaining high performance for dynamic wireless environments.

Shouman Wang and Jiajia Chen, in their work titled “A New Input Grouping and Sharing Method to Design Low Complexity FFT Implementation” [9], present that the Fast Fourier Transform (FFT) is a fundamental algorithm widely used in digital signal processing, wireless communication, and scientific computing applications. The conventional Discrete Fourier Transform (DFT) requires ($O(N^2)$) computations, making it inefficient for large-scale systems, whereas the introduction of the Cooley–Tukey FFT algorithm reduces the computational complexity to ($O(N \log N)$), enabling efficient real-time and hardware implementations. Various FFT architectures, including memory-based and pipelined structures such as Single-path Delay Feedback (SDF), Multi-path Delay Feedback (MDF), and Multi-path Delay Commutator (MDC), have been developed to improve performance; however, pipelined architectures demand higher hardware resources, particularly multipliers and adders. To address these challenges, optimization techniques such as Canonical Signed Digit (CSD) arithmetic, common subexpression elimination, and twiddle factor merging have been introduced to reduce computational complexity. FFT implementations are also categorized into fixed-point and floating-point designs, where fixed-

point architectures are preferred in VLSI systems due to their lower hardware complexity and power consumption. The authors propose a novel low-complexity FFT architecture based on an input grouping method that reduces the number of multiplications and an input partial sum sharing (IPS) technique that minimizes the number of adders, resulting in significant improvements in hardware efficiency, including reductions in area, power, and delay, making the design highly suitable for modern applications such as 5G communication systems and high-speed digital signal processing.

Chapter 3

FFT IMPLEMENTATION

3.1 Existing Method

The Fast Fourier Transform (FFT) is an efficient algorithm used to compute the Discrete Fourier Transform (DFT). In essence, FFT transforms a signal from the time domain into the frequency domain, allowing us to analyze the frequency components of the signal. Multiplication is one of the most resource-intensive arithmetic operations, so reducing the number of multipliers can significantly lower the overall cost. Thanks to the twiddle factor merging technique described in (5), the FFT output is ultimately expressed as a sum of products between twiddle factors and inputs. To optimize this, we propose an input regrouping method. When different inputs are multiplied by twiddle factors with the same absolute value, we can group these inputs together and perform addition or subtraction before multiplying the result by the common twiddle factor. This method uses additional buffers to store inputs that arrive earlier, but the reduction in the number of multipliers offers a more substantial saving in terms of resource usage. For a clearer illustration, consider a 16-point FFT. By recursively applying (2), the computation of a 16-point FFT can be expressed as:

$X(k) = (x(0) + W_k^2 x(8)) + W_k^4 [x(4) + W_k^2 x(12)] + W_k^8 [(x(2) + W_k^2 x(10)) + W_k^4 (x(6) + W_k^2 x(14))] + W_k^{16} [(x(1) + W_k^2 x(9)) + W_k^4 ((x(5) + W_k^2 x(13))) + W_k^8 ((x(3) + W_k^2 x(11)) + W_k^4 ((x(7) + W_k^2 x(15))))]$, where $X(k)$ is the k th output in the frequency domain. Using Euler's formula and the twiddle factor merging from (5), $X(k)$ can be further simplified and expressed as:

$$X(k) = (x(0) + W_k^2 x(8)) + \cos(k\pi/2)[x(4) + W_k^2 x(12)] + \cos(k\pi/4)[x(2) + W_k^2 x(10)] + \cos(3k\pi/4)[x(6) + W_k^2 x(14)] + \cos(k\pi/8)[x(1) + W_k^2 x(9)] + \cos(7k\pi/8)[x(7) + W_k^2 x(15)] + \cos(5k\pi/8)[x(5) + W_k^2 x(13)] + \cos(3k\pi/8)[x(3) + W_k^2 x(11)] - j\{\sin(k\pi/2)[x(4) + W_k^2 x(12)] + \sin(k\pi/4)[x(2) + W_k^2 x(10)] + \sin(3k\pi/4)[x(6) + W_k^2 x(14)] + \sin(k\pi/8)[x(1) + W_k^2 x(9)] + \sin(7k\pi/8)[x(7) + W_k^2 x(15)] + \sin(5k\pi/8)[x(5) + W_k^2 x(13)] + \sin(3k\pi/8)[x(3) + W_k^2 x(11)]\} \quad (6)$$

Here, W_k^2 is equal to 1 or -1 , and multiplying the input by -1 or 1 can be implemented directly by hardwiring. Using the symmetry properties of trigonometric functions, we have:

$$\begin{aligned} \cos(k\pi/4) &= W_k^2 \cos(3k\pi/4), \\ \cos(k\pi/8) &= W_k^2 \cos(7k\pi/8), \\ \cos(3k\pi/8) &= W_k^2 \cos(5k\pi/8), \\ \sin(k\pi/4) &= -W_k^2 \sin(3k\pi/4), \\ \sin(k\pi/8) &= -W_k^2 \sin(7k\pi/8), \\ \sin(3k\pi/8) &= -W_k^2 \sin(5k\pi/8). \end{aligned}$$

Definition 1: An IPS matrix is a triangular matrix with $g - 1$ rows and $g - 1$ columns. The entry $IPS(m, n-1)$ in the m th row and the $(n-1)$ th column represents the number of occurrences of a weight-two IPS formed by a_m and a_n , where $m \in [1, g-1]$, $n \in [2, g]$. Since we approximate the

cost of a subtractor to be the same as that of an adder, $a_j - a_i$ and $a_i - a_j$ are regarded as the same negative weight-two IPS. Therefore, these do not need to be distinguished. Moreover, since $a_i + a_j$ and $a_i - a_j$ can be computed using a single unified adder/subtractor (UAS) operator [22], which has lower cost than using both an adder and a subtractor, the entries in the IPS matrix store the sum of positive weight-two IPSs ($a_i + a_j$) and negative weight-two IPSs ($a_i - a_j$). For example, if there are five inputs in the group, and three high-weight IPSs exist ($IPS1 = a_1 + a_2 + a_3 + a_4 + a_5$, $IPS2 = a_1 - a_2 + a_4 + a_5$, $IPS3 = a_1 - a_2 - a_3 + a_4$), the matrix will account for these common partial sums to minimize the adder cost.

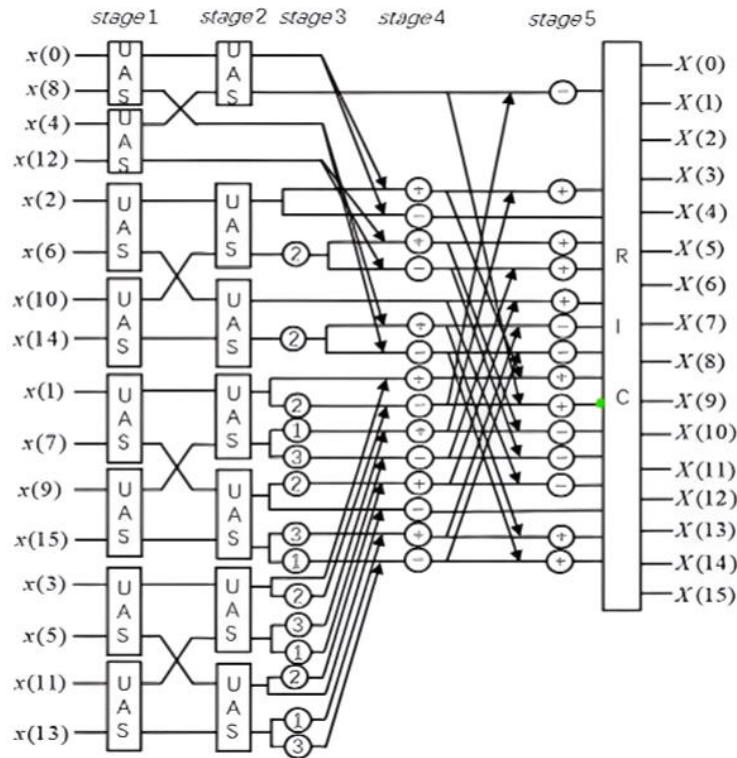


Fig.3.1. The Existing 16-point FFT architecture.

With the proposed algorithm, the 16-point approximated FFT architecture is depicted in Fig., 1, 2 and 3 represent multiplications with $\cos(\pi/8)$, $\sin(\pi/4)$ and $\sin(\pi/8)$ respectively. Real-imaginary-combination (RIC) block is to list the outputs.

After all inputs in the same group are paired, the unique paired weight-two IPSs can be treated as the new basis, similar to the individual inputs in the weight-two IPSs. Subsequently, these weight-two IPSs are used to construct higher-weight IPSs that can be shared. A similar IPS matrix is constructed to store the number of occurrences of these higher-weight IPSs. The search and selection process for these higher-weight IPSs follows the same algorithm used for searching and selecting the weight-two IPSs. This process continues iteratively, with the search for higher-weight IPSs, until a weight- g IPS is constructed, representing the full set of inputs in the group.

For illustration, using the previous example, a complete binary tree,, is constructed. IPS1, IPS2, and IPS3, which represent three high-weight IPSs, serve as the root vertices, with the edges representing the signs. M1, M2, and M3 are intermediary vertices. Paired weight-two IPSs such

as $(a1 + a4)$ and $(a2 + a3)$ are formed. The fixed weight-three IPS $(a1 + a4 + a5)$ can be shared by IPS1 and IPS2. outlines the flow for the proposed IPS sharing method. The variable h represents the IPS weight and is initially set to 2, while g is the number of inputs in the group. Weight-two IPSs are paired based on their probabilities until the IPS matrix becomes empty. If h is still less than g , the process continues to search for higher-weight IPSs until the full set of inputs in the group is efficiently shared.

3.1.1 Input Data (X(1) to X(31)):

The 32 input data points, labeled $X(0)$ to $X(31)$, represent the sampled time-domain values of the signal that is being analyzed. Each of these values corresponds to a specific time sample of a continuous signal, typically representing an audio signal, sensor data, or any discrete-time series data.

3.1.2 Input Grouping:

Input Grouping is a technique used in FFT architecture to reduce the number of multiplications by grouping input signals that are multiplied by twiddle factors with the same magnitude. The grouped inputs are first added or subtracted, and then the result is multiplied by the common twiddle factor, which reduces the number of required multipliers and lowers hardware complexity.

3.1.3 Partial Sum Sharing:

Input Partial Sum Sharing (IPS Sharing) is a hardware optimization method where common intermediate sums or differences of grouped inputs are reused to reduce the number of adders required in FFT computation

3.1.4 UAS Units in the Architecture:

The UAS (likely a logic unit or block) shown in the diagram is used for performing specific operations on the input data. In the context of FFT, UAS could represent an arithmetic or data manipulation unit that performs addition, subtraction, or complex multiplication operations. These operations are essential for combining input data points in the butterfly structure of the FFT.

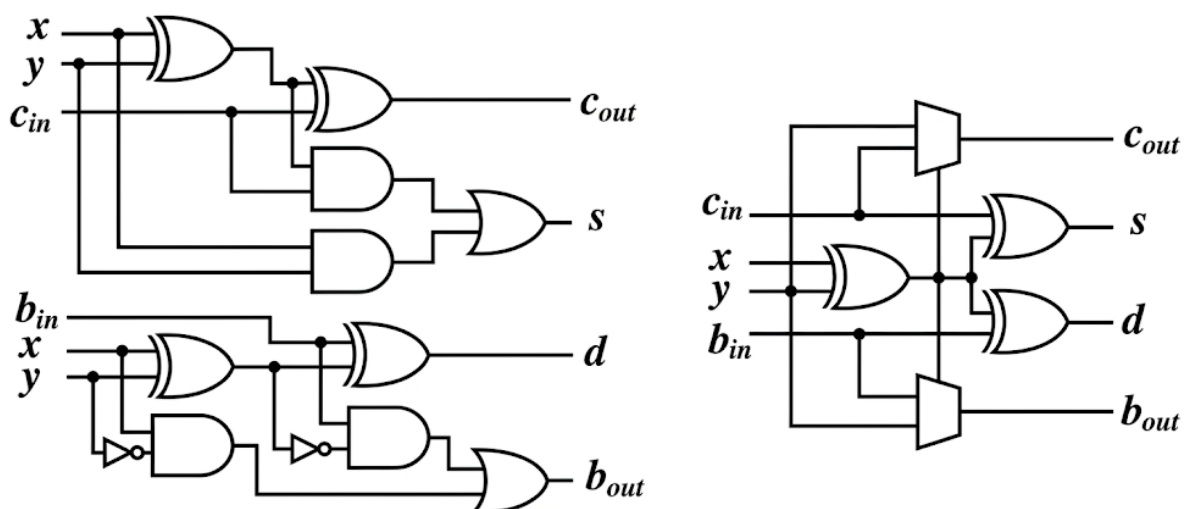


Fig.3.2. UAS Architecture

3.1.5 Butterfly Structure:

One of the fundamental aspects of the FFT algorithm is its use of the butterfly structure. This structure combines pairs of data points in each stage of the FFT computation. Each stage involves computing complex values (real and imaginary parts) for each pair, with the outputs being passed on to the next stage. The butterfly operation helps in reducing the computational complexity from $O(N^2)$ to $O(N \log N)$.

3.1.6 Stages of the FFT:

The FFT is typically computed in a series of stages, each one reducing the problem size. For a 32-point FFT, there are 5 stages, as $\log_2(32) = 5$. Each stage involves the pairing of data points and applying the butterfly operation to reduce the total number of computations required.

3.1.7 Real, Imaginary, and Complex Outputs:

The diagram shows outputs labeled R, I, and C. These correspond to the real and imaginary parts of the FFT results. R is the real component, I is the imaginary component, and C represents the combined complex values. These outputs are essential for fully representing the frequency components of the signal.

3.1.8 Twiddle Factors:

One of the key components of the FFT is the use of twiddle factors—complex exponential terms that are used to scale the data during the butterfly operations. These factors help in rotating the data in the complex plane, which is an essential part of transforming the data from the time domain to the frequency domain.

3.1.9 Data Flow and Connectivity:

The arrows in the diagram indicate the flow of data between different blocks in the architecture. The data starts at the inputs ($X(0)$ to $X(31)$), flows through several processing stages (UAS units and butterfly operations), and ultimately reaches the outputs (R, I, and C). This represents the transformation of the time-domain signal to the frequency domain through successive stages.

3.1.10 Processing of Even and Odd Data Points:

A common technique in FFT algorithms is to separate the data into even and odd indexed points. These two sets of data are then processed separately in the initial stages of the FFT. The diagram may suggest such a separation, where even and odd indices are processed in different branches of the butterfly structure, leading to more efficient computation.

3.1.11 Recursive Computation in FFT:

The 32-point FFT algorithm recursively divides the data set into smaller sub-problems. In each stage, the problem size is halved, and the results of smaller FFTs are combined to generate the final FFT result. This recursive breakdown is a key reason for the speedup provided by FFT compared to a direct DFT computation.

3.1.12 Optimization of FFT through In-Place Computation:

In many FFT implementations, the data is transformed in-place, meaning that the input data array is reused for storing intermediate results. This helps in reducing memory usage and improving the overall efficiency of the algorithm, particularly in hardware or embedded systems where memory is limited.

3.1.13 Handling of Complex Numbers:

The FFT involves operations on complex numbers, where each data point has both a real and imaginary component. The diagram highlights the importance of separating and managing these components—R for real, I for imaginary, and C for the combined result. Handling complex numbers efficiently is one of the challenges in implementing the FFT algorithm.

3.1.14 Importance of the FFT in Signal Processing:

The FFT is widely used in many fields, including signal processing, communications, audio analysis, and image processing. It is particularly useful for analyzing the frequency content of a signal, detecting patterns, filtering noise, or transforming data into a more usable form for further processing.

3.1.15 Parallelization in FFT Computation:

Due to the recursive nature of FFT, it is well-suited for parallel computation. In modern systems, FFTs are often implemented in parallel to further speed up the computation. This can be achieved using multi-core processors, GPUs, or distributed computing techniques.

3.1.16 Handling of Input and Output Data in FFT:

The input data to the FFT is usually a set of time-domain samples that represent a signal. The output is a set of frequency-domain coefficients that indicate how much of each frequency component is present in the original signal. The separation of real and imaginary components in the output allows for a complete representation of these frequency components.

3.1.17 Efficiency of FFT Over Direct DFT:

Direct computation of the Discrete Fourier Transform (DFT) requires $O(N^2)$ operations, which makes it computationally expensive for large data sets. The FFT reduces this to $O(N \log N)$ operations, making it a much more efficient algorithm, especially for large-scale data or real-time applications.

3.1.18 Twiddle Factor Optimization:

The twiddle factors in FFT are periodic, and this periodicity can be exploited to reduce the number of unique values needed in the computation. In many implementations, twiddle factors are pre-computed and stored, saving computation time during the FFT process.

3.1.19 Memory Considerations in FFT Implementation:

In hardware implementations, memory usage is an important consideration. The architecture depicted in the diagram might optimize memory usage by reusing the same memory locations for different stages of the computation. This is especially important in embedded systems or FPGA implementations where memory resources are limited.

3.1.20 Hardware Implementation of FFT:

The diagram suggests a hardware-oriented implementation of the FFT, where the data is processed using dedicated logic blocks. This can be implemented in digital hardware such as FPGAs or ASICs for high-speed processing, making it suitable for real-time signal processing applications. The use of parallel computation and in-place memory storage is crucial for achieving high throughput in hardware implementations.

Chapter 4

PROPOSED ARCHITECTURE AND METHOD

4.1 Proposed 32-point Architecture

In this work, a novel 32-point Fast Fourier Transform (FFT) architecture is developed by using a 16-point FFT as the core computational unit, which serves as the fundamental building block of the design. Instead of implementing a complete 32-point FFT directly—which would require a higher number of arithmetic operations, increased hardware resources, and greater power consumption—the proposed approach decomposes the overall computation into two smaller and more manageable 16-point FFT operations. This decomposition significantly simplifies the design by reducing the number of complex multiplications and additions required, thereby lowering computational complexity and improving overall hardware efficiency.

This approach not only reduces design complexity but also enables efficient utilization of hardware resources such as adders, multipliers, and memory elements. Furthermore, since the 16-point FFT module is already optimized using techniques like input grouping and partial sum sharing, the overall 32-point FFT inherits these advantages, resulting in improved performance. However, during implementation, it is observed that the power consumption increases due to continuous clock switching activity in the circuit. To overcome this issue, we proposed a clock gating technique.

4.2 Proposed Method

4.2.1 Flip flop-based clock gating technique:

In many applications, latch based designs are moved to flip flop-based designs. By splitting flip flop, we can see two latches from the master slave theorem. In this technique, we can see D flip flop with AND gate Flip-flop-based clock gating is a technique used to reduce dynamic

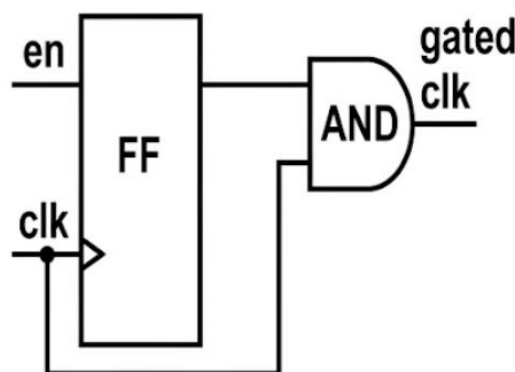


Fig.4.1. Flip flop-based clock gating technique

power consumption in digital circuits by disabling the clock when the system is idle. Unlike latch-based clock gating, which can introduce glitches, this approach ensures a stable and glitch-free gated clock by synchronizing the enable signal (EN) with the clock edge using a flip-flop. The technique works by sampling the enable signal on the rising edge of the clock using a D flip-flop.

The output of the flip-flop (en_ff) is then ANDed with the clock signal to generate a gated clock (CLK_GATED). This ensures that the gated clock transitions only occur at well-defined clock edges, preventing glitches and ensuring reliable operation.

4.2.1.1 Clock Gating Truth Table

CLOCK	ENABLE	FF OUTPUT	GATED CLOCK	OPERATION
0	0	0	0	Clock OFF
1	0	0	0	Clock OFF
0	1	1	0	Waiting
1	1	1	1	Clock ON

Table.4.1. Clock Gating Truth Table

From the above figure, gated clock goes to high when flip flop output and clock are in high state otherwise gated clock goes to zero state. That means when clock in sleep mode then gated clock also in zero state This implementation ensures that the enable signal is sampled at every rising edge of the clock, preventing glitches and hold-time violations. The technique is widely used in low-power processors, digital signal processing circuits, and FPGA/ASIC designs where power efficiency is a priority. Compared to latch-based clock gating, the flip-flop approach provides better timing closure, prevents race conditions, and is easier to integrate into synchronous designs

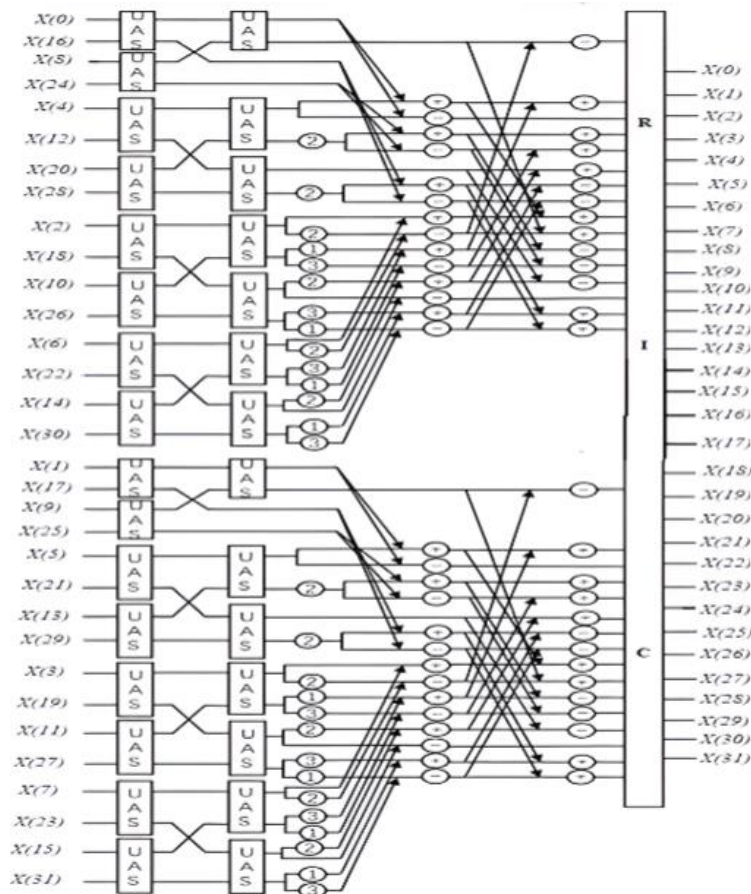


Fig.4.2. The proposed 32-point FFT architecture

4.3 Implementation of Proposed Architecture using Verilog HDL

The proposed 32-point FFT architecture along with the flip-flop based clock gating technique is implemented using Verilog Hardware Description Language (HDL). The design follows a modular approach, where each functional block is described as an independent module and later integrated to form the complete system. This modular implementation improves code reusability, scalability, and ease of debugging.

The top-level module (top.v) acts as the main controller that connects all submodules and manages the overall data flow. It coordinates the input signals, clock, and enable signals required for proper system operation. The 32-point FFT module (top_32.v) is designed using two 16-point FFT blocks based on the divide-and-conquer approach. The input data is divided into even and odd sequences, and each sequence is processed independently using the 16-point FFT core.

The Unit Arithmetic Structure module (UAS.v) is used to perform arithmetic operations such as addition and subtraction required for butterfly computations. It supports both real and imaginary components of the FFT, ensuring correct complex number processing. The multiplexer module (mux_2to1.v) is used to control the selection of input data paths during different stages of computation, enabling efficient resource sharing.

To reduce power consumption, a flip-flop-based clock gating module (flipflop_based_clk.v) is implemented. This module controls the clock signal using an enable signal. The clock is activated only during computation and remains disabled during idle conditions, thereby reducing unnecessary switching activity and dynamic power consumption.

The complete design is simulated and verified using Xilinx Vivado. RTL schematics are generated to validate the hardware structure, and simulation waveforms are analyzed to ensure correct functionality. The integration of clock gating with the FFT architecture demonstrates improved power efficiency while maintaining performance and accuracy.

4.4 RTL Schematic of Proposed Architecture

The Register Transfer Level (RTL) schematic of the proposed 32-point FFT architecture is generated using Xilinx Vivado after synthesis of the Verilog design. The RTL schematic represents the high-level structural view of the design, showing how different modules such as the 16-point FFT blocks, Unit Arithmetic Structure (UAS), multiplexers, and clock gating unit are interconnected.

From the RTL schematic, it can be observed that the design follows a modular architecture, where the 32-point FFT is implemented using two 16-point FFT modules. The input signals are distributed to these modules, and the outputs are combined through structured interconnections. The data flow between different modules is clearly represented, showing how signals propagate through various stages of computation.

The clock gating block is also integrated into the RTL design, where the clock signal is controlled

using an enable signal. This ensures that the clock is active only during computation. The RTL schematic verifies that the design is logically correct and follows the intended architecture. It provides a clear understanding of module hierarchy, signal flow, and interconnections before moving to lower-level implementation.

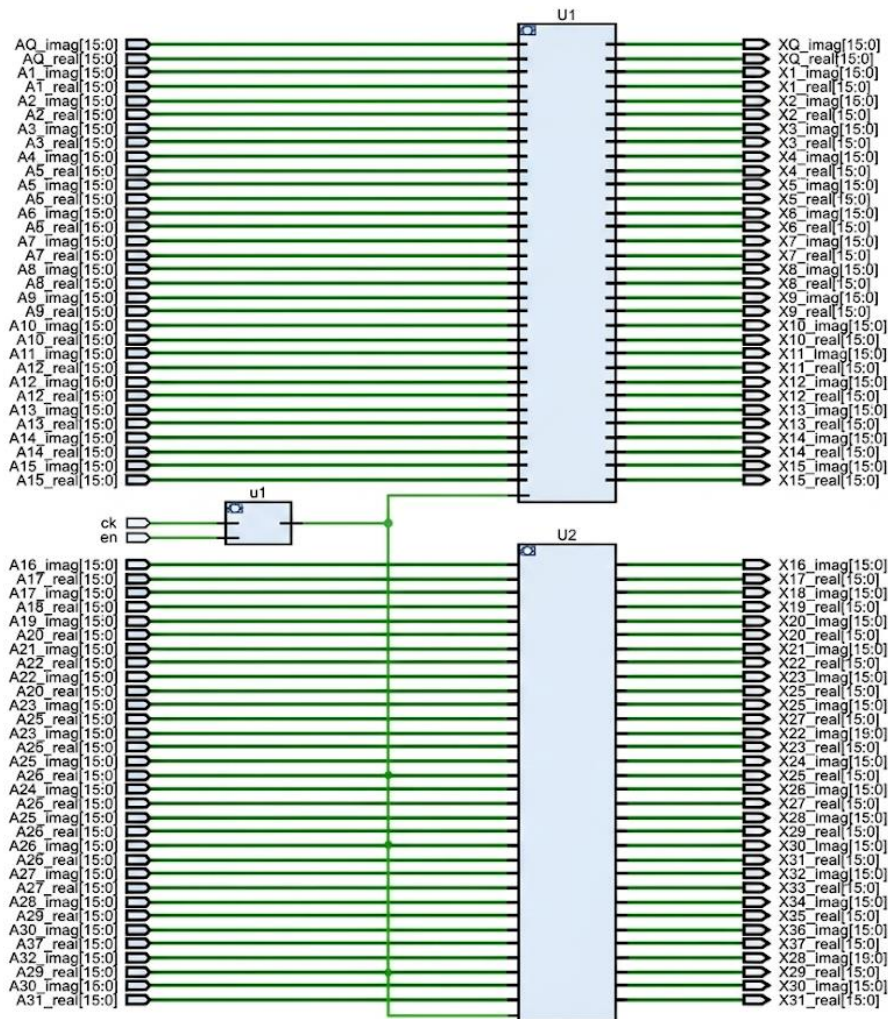


Fig.4.3. RTL Schematic of Proposed 32-point FFT Architecture

4.5 Technology Schematic of Proposed Architecture

The technology schematic represents the gate-level implementation of the proposed design after synthesis. Unlike the RTL schematic, which shows a high-level view, the technology schematic illustrates the actual hardware components such as logic gates, flip-flops, multiplexers, and interconnections used to implement the design on FPGA or ASIC.

From the technology schematic, it is evident that the design consists of a large number of interconnected logic elements, including LUTs (Look-Up Tables), flip-flops, and routing paths. The complexity of the schematic indicates the detailed mapping of the FFT operations into hardware resources. The arithmetic operations, control logic, and clock gating circuitry are all implemented using basic digital components.

The flip-flop-based clock gating logic is clearly reflected in the technology schematic, where the

clock signal is ANDed with the synchronized enable signal to generate a gated clock. This helps in reducing switching activity and dynamic power consumption at the hardware level.

The technology schematic validates that the design is successfully synthesized and mapped onto hardware resources. It also helps in analyzing parameters such as area utilization, timing performance, and power consumption. This level of representation is essential for understanding the actual physical implementation of the proposed FFT architecture.

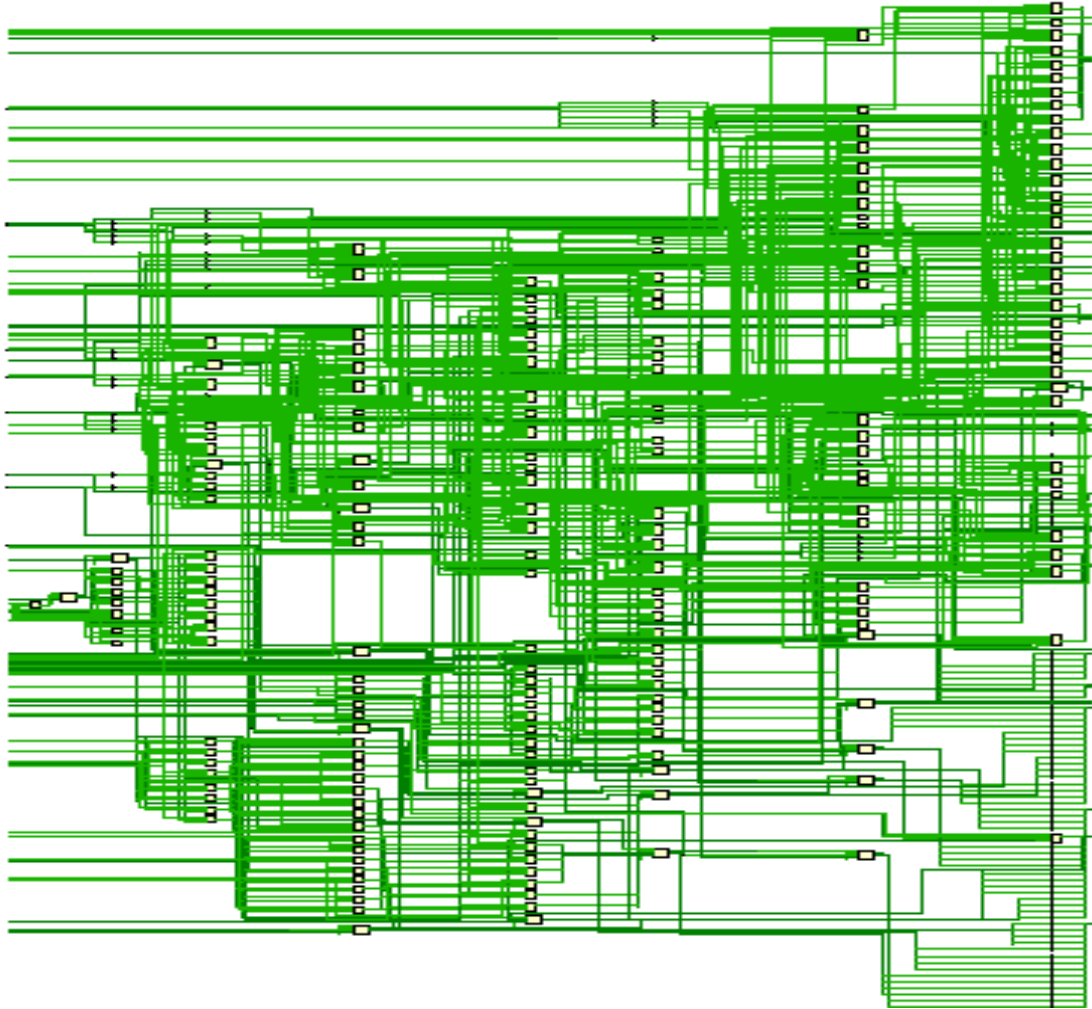


Fig.4.4. Technology Schematic of Proposed 32-point FFT Architecture

Chapter 5

XILINX VIVADO DESIGN ENVIRONMENT

The Xilinx Vivado Design Environment is a powerful Electronic Design Automation (EDA) tool used for designing, simulating, synthesizing, and implementing digital systems on FPGA devices. It provides an integrated platform that supports Hardware Description Languages (HDLs) such as Verilog and enables designers to develop efficient hardware systems.

Vivado allows designers to follow a structured design flow starting from design entry to final implementation. The design is described using Verilog HDL at different abstraction levels such as behavioral and Register Transfer Level (RTL). The tool provides simulation capabilities that help in verifying the functionality of the design by applying input test signals and analyzing output waveforms. This ensures that the design works correctly before moving to hardware implementation.

After simulation, the design is synthesized, where the Verilog code is converted into a hardware representation using components like Look-Up Tables (LUTs), flip-flops, and multiplexers. The RTL schematic generated during this stage provides a clear view of module connections and data flow.

The next stage is implementation, which includes mapping, placement, and routing of the design onto the FPGA. The technology schematic generated at this stage represents the actual hardware realization of the design. Vivado also provides reports for timing analysis, power estimation, and resource utilization, which help in evaluating the performance of the design.

Overall, the Vivado Design Environment simplifies the development process and enables efficient implementation of complex digital systems like the proposed 32-point FFT architecture with clock gating.

5.1 Benefits

- i. **Integrated Design Environment:** Vivado provides a complete platform for design entry, simulation, synthesis, and implementation in a single tool, reducing complexity.
- ii. **Functional Verification:** It allows behavioral simulation to verify the correctness of the design before hardware implementation.
- iii. **RTL & Technology Schematics:** Generates RTL and technology schematics to visualize design structure and hardware mapping.
- iv. **Performance Analysis:** Provides detailed reports for area, delay, and power, helping in evaluating design efficiency.
- v. **Debugging Support:** Includes advanced debugging tools to easily identify and correct design

errors.

vi. **FPGA Implementation:** Supports generation of bitstream files for direct implementation on FPGA devices.

vii. **Improved Productivity:** Reduces development time and enhances design efficiency through automation and integration.

5.2 Features

The Xilinx Vivado Design Suite provides a wide range of advanced features for efficient digital system design and verification. It supports multiple design flows, including RTL design, simulation, synthesis, and implementation, all within a single integrated environment. This enables designers to move seamlessly through different stages of the design process. Vivado also supports hierarchical design, allowing complex systems to be developed by integrating multiple modules in a structured manner.

Another important feature of Vivado is its ability to generate RTL and technology schematics. The RTL schematic provides a high-level view of the design, showing module interconnections and data flow, while the technology schematic represents the actual gate-level implementation using hardware components. These visual representations help designers better understand and analyze the design.

Vivado also provides powerful analysis tools such as timing analysis, power estimation, and resource utilization reports. These features help in evaluating the performance, speed, and efficiency of the design. In addition, the tool offers an interactive debugging environment where designers can monitor signals, analyze waveforms, and validate the design functionality. Overall, these features make Vivado a comprehensive and efficient tool for implementing complex digital systems.

5.3 Vivado Design Flow: Project Creation, Synthesis, Implementation, and Simulation

5.3.1 Creating a New Project

Creating Projects, you can use the New Project wizard to easily create different types of projects in the Vivado IDE. To open the New Project wizard, select File > New Project. This wizard enables you to specify a project location and name and create the types of projects shown in below figure.5.1.

5.3.1.1 Project Name and Location

Project Name: Write the name of your new project which is user defined.

Project Location: The directory where you want to store the new project in the specified location in one of your drive. In above window they are stored in location c drive which is not correct, the location of software and code should not be same location and Clicking on NEXT.

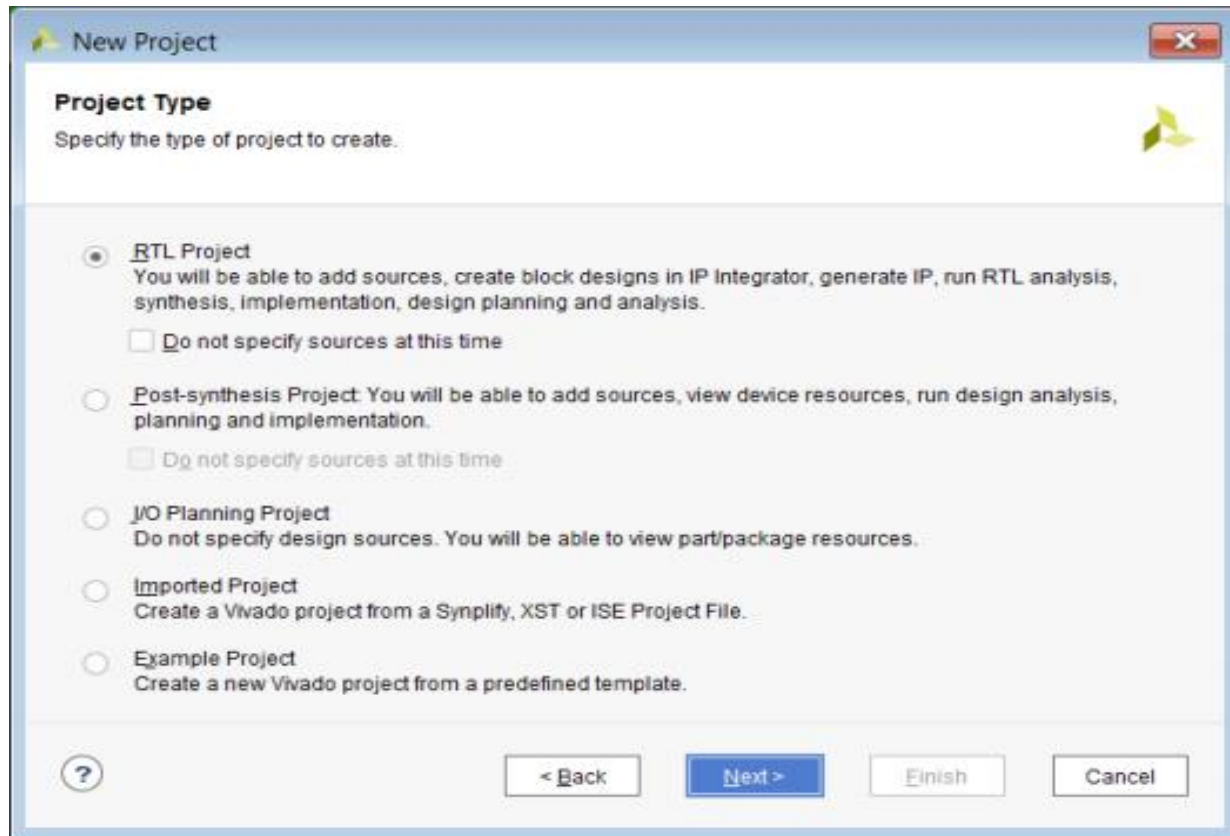


Fig.5.1. New Project Wizard—Project Type Page

5.3.1.2 Device Selection

- **Device Family:** Family of the FPGA/CPLD used. In this laboratory we will be using the Artix - 7 FPGAs.
- **Device:** The number of the actual device. For this lab you may enter **XC7A200TFBG676-2** (this can be found on the attached prototyping board)
- **Package:** The type of package with the number of pins. The Spartan FPGA used in this lab is packaged in FBG676 package.
- **Speed Grade:** The Speed grade is “-2”.
- **Synthesis Tool:** XST [VHDL/Verilog]
- **Simulator:** The tool used to simulate and verify the functionality of the design. Then click on **NEXT** to save the entries.

5.3.2 Opening Designs:

Use the Flow Navigator or Flow menu to select the following commands:

- Open Elaborated Design
- Open Synthesized Design
- Open Implemented Design

The Flow > Open Implemented Design command populates the Vivado IDE as shown in below figure.

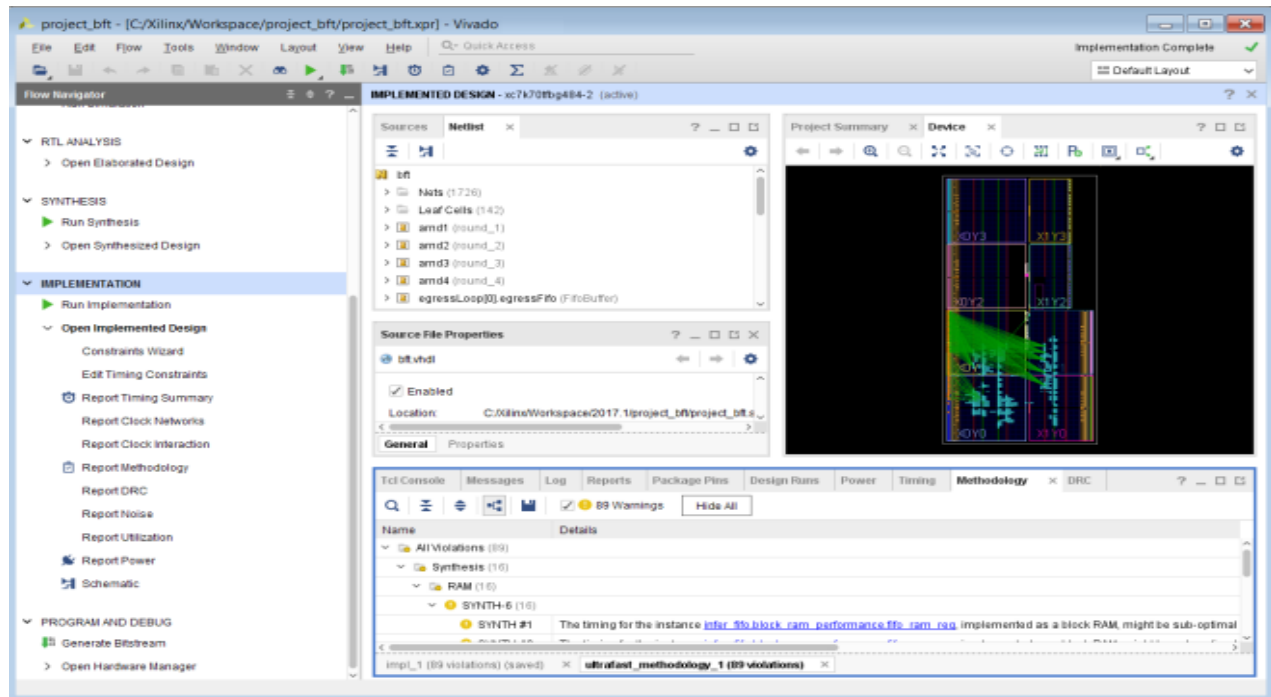


Fig.5.2. Implemented design View in Vivado Design Suite

5.3.3 Creating Verilog HDL Source File

- All project files such as schematics, netlists, Verilog files, VHDL files, etc., will be stored in a subdirectory with the project name.
- In order to open an existing project in Xilinx Tools, select **File->Open Project** to show the list of projects on the machine. Choose the project you want and click **OK**.
- If creating a new source file, click on the **NEW SOURCE**.
- Creating a Verilog HDL input file for a combinational logic design:
- In this lab we will enter a design using a structural or RTL description using the Verilog HDL. You can create a Verilog HDL input file (.v file) using the HDL Editor available in the Xilinx Vivado Tools (or any text editor).
- In the previous window, click on the **NEW SOURCE**
- (Note: “**Add to project**” option is selected by default. If you do not select it then you will have to add the new source file to the project manually.)
- Select **Verilog Module** and in the “File Name:” area, enter the name of the Verilog source file you are going to create. Also make sure that the option **Add to project** is selected so that the source need not be added to the project again. Then click on **Next** to accept the entries.
- In the **Port Name** column, enter the names of all input and output pins and specify the **Direction** accordingly. A Vector/Bus can be defined by entering appropriate bit numbers in the **MSB/LSB** columns. Then click on **Next>** to get a window showing all the new source information above window. If any changes are to be made, just click on **<Back** to go back and make changes. If everything is acceptable, click on **Finish > Next > Next > Finish** to continue.
- Once you click on **Finish**, the source file will be displayed in the sources window in the

Project Navigator. If a source has to be removed, just right click on the source file in the **Sources in Project** window in the **Project Navigator** and select **remove** in that. Then select **Project -> Delete Implementation Data** from the Project Navigator menu bar to remove any related files.

5.3.4 Editing the Verilog source file

The source file will now be displayed in the **Project Navigator** window (Figure 8). The source file window can be used as a text editor to make any necessary changes to the source file. All the input/output pins will be displayed. Save your Verilog program periodically by selecting the **File->Save** from the menu. You can also edit Verilog programs in any text editor and add them to the project directory using “Add Copy Source”.

Here in the above window we will write the Verilog programming code for specified design and algorithm in the window.

After writing the programming code we will go for the synthesis report.

5.3.5 Configuring Project Settings

You can configure the Project Settings in the Settings dialog box to meet your design needs. These settings include general settings, related to the top module definition and language options, as well as simulation, elaboration, synthesis, implementation, bitstream, and IP settings.

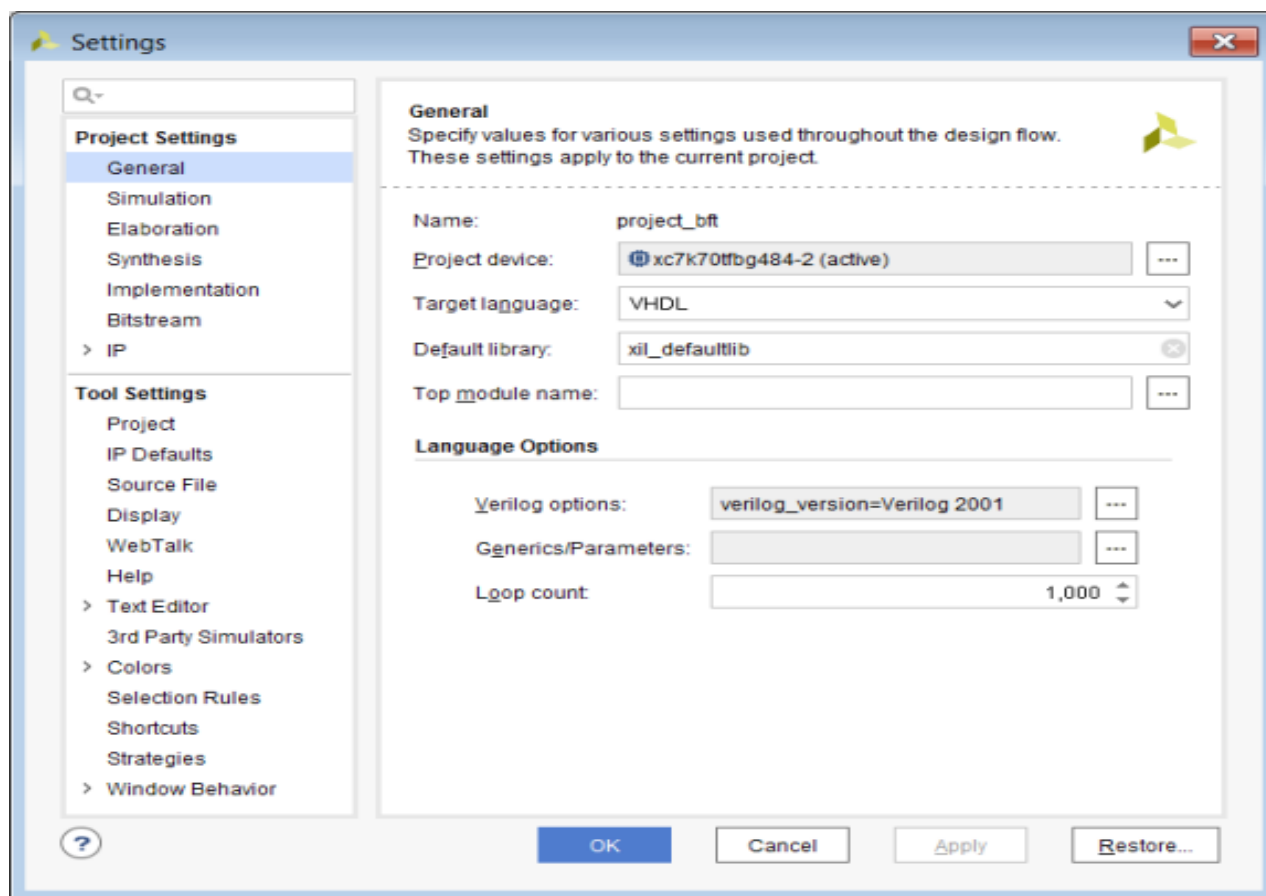


Fig.5.3. Project Setting Window

Settings Dialog Box—Project Settings General Category

To open the Settings dialog box, use any of the following methods:

- In the Flow Navigator Project Manager section, click Settings.
- Select Tools > Settings.
- In the main toolbar, click the Settings toolbar button .
- In the Project Summary, click the Edit link next to Settings.

5.3.6 Synthesis and Implementation of the Design:

The design has to be synthesized and implemented before it can be checked for correctness, by running functional simulation or downloaded onto the prototyping board. With the top-level Verilog file opened (can be done by double-clicking that file) in the HDL editor window in the right half of the Project Navigator, and the view of the project being in the **Module view** , the **implement design** option can be seen in the **process view**. **Design entry utilities** and **Generate Programming File** options can also be seen in the process view. To synthesize the design, double click on the **Synthesize Design** option in the **Processes window**. To implement the design, double click the **Implement design** option in the **Processes window**. It will go through steps like **Translate, Map and Place & Route**. If any of these steps could not be done or done with errors, it will place a **X** mark in front of that, otherwise a tick mark will be placed after each of them to indicate the successful completion. After synthesis right click on synthesis and click view text report in order to generate the report of our design.

5.3.7 XILINX VIVADO SIMULATION PROCEDURE:

After completion of synthesis, we will go simulation in order to verify the functionality of the implemented design.

Click on **Run Simulation** and set the module that is need to Run

Next **double click on Run Behavioral Simulation** to check the errors. If no errors are found then double click on simulate behavioral model to get the output waveforms.

After clicking on **simulate behavioral model**, the simulation widow will appear pass the input values by making force constant and if it is clock by making force clock. Mention the simulation period and run for certain time and results will appear as shown in following window. Verify the results to the given input values.

Chapter 6

RESULTS

6.1 32 – POINT FFT WITHOUT CLOCK GATING RESULTS

6.1.1 Simulation Waveform:

The simulation waveform represents the functional verification of the 32-point FFT architecture without clock gating. The input signals consist of real and imaginary components (A0_real, A0_imag, ...), and the corresponding output signals (X15_real, X15_imag, etc.) are observed. The waveform shows stable transitions and correct propagation of input data through different stages of the FFT. Since clock gating is not used, the clock signal continuously drives all registers, resulting in constant switching activity throughout the simulation period. The outputs obtained confirm the correct operation of the FFT design.

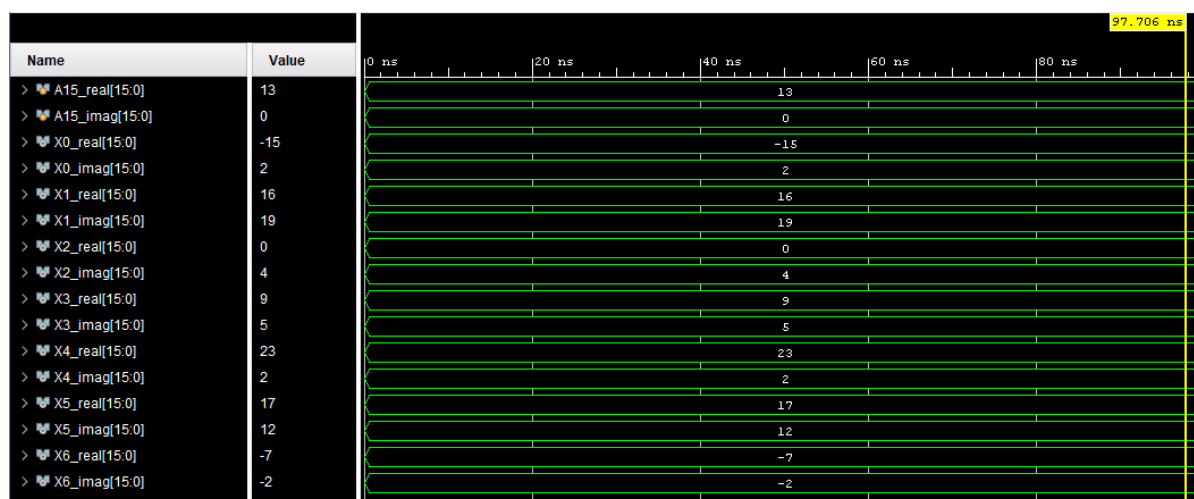


Fig.6.1. Simulation of 32-Point FFT Architecture Without Clock Gating

6.1.2 Area Utilization:

The area report indicates the hardware resources utilized by the design. The 32-point FFT architecture consumes approximately 3290 Slice LUTs and 2048 Bonded IOBs. Since clock gating is not implemented, there is no optimization in register usage or logic reduction, leading to relatively higher resource utilization.

Name	Slice LUTs (134600)	Bonded IOB (400)
N top_32	3290	2048

Fig.6.2. Device Utilization

6.1.3 Delay Analysis:

The delay analysis report shows the timing performance of the 32-point FFT design. The critical path delay observed is approximately **10.561 ns**, which includes logic delay (~5.610 ns) and net delay (~4.951 ns). The slack value is positive, indicating that the design meets timing requirements. However, due to the absence of clock gating, unnecessary switching contributes indirectly to timing inefficiencies in larger designs.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	∞	13	11	33	A11_imag[3]	X15_imag[15]	10.561	5.610	4.951
↳ Path 2	∞	13	11	33	A27_imag[3]	X23_imag[15]	10.561	5.610	4.951
↳ Path 3	∞	13	11	33	A27_imag[3]	X31_imag[15]	10.561	5.610	4.951
↳ Path 4	∞	13	11	33	A11_imag[3]	X7_imag[15]	10.561	5.610	4.951
↳ Path 5	∞	13	11	33	A11_imag[3]	X15_imag[13]	10.556	5.605	4.951
↳ Path 6	∞	13	11	33	A27_imag[3]	X23_imag[13]	10.556	5.605	4.951
↳ Path 7	∞	13	11	33	A27_imag[3]	X31_imag[13]	10.556	5.605	4.951
↳ Path 8	∞	13	11	33	A11_imag[3]	X7_imag[13]	10.556	5.605	4.951
↳ Path 9	∞	13	11	33	A11_imag[3]	X15_imag[14]	10.496	5.545	4.951
↳ Path 10	∞	13	11	33	A27_imag[3]	X23_imag[14]	10.496	5.545	4.951

Fig.6.3. Timing Analysis Report

6.1.4 Power Consumption:

The power analysis report shows that the total on-chip power consumption is approximately **1000.115 W**, which is significantly high. The junction temperature reaches **125°C**, indicating thermal issues. The absence of clock gating leads to continuous clock activity across all modules, causing excessive dynamic power consumption. This highlights the major drawback of the existing design and justifies the need for power optimization techniques like clock gating.

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	1000.115 W (Junction temp exceeded!)
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	125.0°C
Thermal Margin:	-1809.8°C (-966.8 W)
Effective SJA:	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

Fig.6.4. Power Analysis Report

6.2 32– POINT FFT WITH CLOCK GATING RESULTS

6.2.1 Simulation Waveform:

The simulation waveform of the proposed 32-point FFT architecture demonstrates the functionality of the design with clock gating enabled. The presence of the enable (en) signal controls the clock activity, allowing the circuit to operate only when required. Initially, undefined states (X) are observed, which later settle to valid values after proper signal propagation. Compared to the existing design, switching activity is reduced because the clock is gated when the enable signal is inactive. This confirms correct functional behavior along with improved power efficiency.

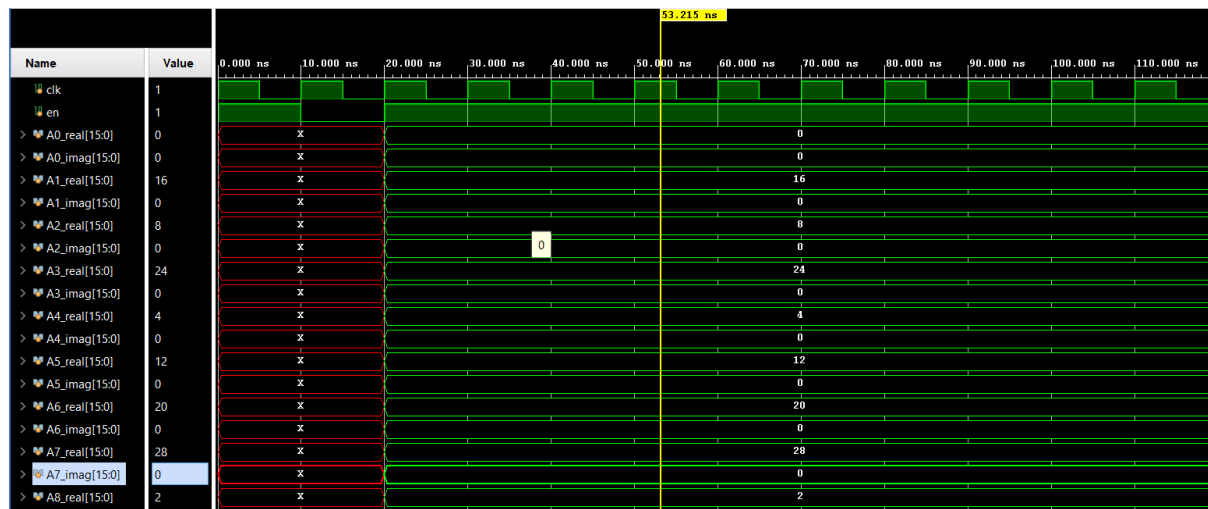


Fig.6.5. Simulation Waveform of 32-Point FFT with Clock Gating

6.2.2 Area Utilization:

The area utilization report indicates that the proposed design uses approximately 3307 Slice LUTs and 897 Slice Registers. Compared to the existing design, there is a slight increase in LUT usage due to additional clock gating logic. However, this increase is minimal and acceptable considering the significant power savings achieved.

Name	Slice LUTs (134600)	Slice Registers (269200)
top_32	3307	897
f1 (flipflop_based_clk)	0	1
u1 (top_16)	1504	448
u2 (top_16_0)	1504	448

Fig.6.6. Device Utilization

6.2.3 Delay Analysis:

The delay analysis of the proposed design shows an improved critical path delay of approximately 7.406 ns, which is significantly lower than the existing design (~10.561 ns). The logic delay is around 3.100 ns, and net delay is 4.306 ns. The reduction in delay is due to optimized switching activity and better control over signal propagation using clock gating. The timing requirements are successfully met with positive slack.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[13]/D	7.406	3.100	4.306
↳ Path 2	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[13]/D	7.406	3.100	4.306
↳ Path 3	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[15]/D	7.404	3.098	4.306
↳ Path 4	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[15]/D	7.404	3.098	4.306
↳ Path 5	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[14]/D	7.343	3.037	4.306
↳ Path 6	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[14]/D	7.343	3.037	4.306
↳ Path 7	∞	12	9	33	A11_imag[3]	u1/X7_imag_reg[12]/D	7.320	3.014	4.306
↳ Path 8	∞	12	9	33	A27_imag[3]	u2/X7_imag_reg[12]/D	7.320	3.014	4.306
↳ Path 9	∞	11	9	33	A11_imag[3]	u1/X7_imag_reg[9]/D	7.308	3.002	4.306
↳ Path 10	∞	11	9	33	A27_imag[3]	u2/X7_imag_reg[9]/D	7.308	3.002	4.306

Fig.6.7. Timing Analysis Report

6.2.4 Power Consumption:

The power analysis report shows that the total on-chip power consumption is reduced to approximately **62.447 W**, which is a significant improvement compared to the existing design (~1000 W). Although the junction temperature still reaches **125°C**, the overall power consumption is drastically minimized due to reduced switching activity. Clock gating effectively disables inactive modules, thereby reducing dynamic power consumption.

Total On-Chip Power:	62.447 W (Junction temp exceeded!)
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	125.0°C
Thermal Margin:	-56.7°C (-29.1 W)
Effective θJA:	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Fig.6.8. Power Analysis Report

6.3 Comparative Analysis of 32-Point FFT with and without Clock Gating:

SI.NO	METHOD	AREA (LUT)	POWER(w)	DELAY(ns)
1	WITHOUT CLOCK	3290	1000.115	10.561
2	WITH CLOCK	3307	62.447	7.406

Table.6.1 Comparative Analysis of 32-Point FFT with and without Clock Gating

The comparison between the existing 32-point FFT design (without clock gating) and the proposed design (with clock gating) is carried out based on key performance parameters such as area, power consumption, and delay.

From the results, it is observed that the **area utilization** slightly increases from **3290 LUTs to 3307 LUTs** in the proposed design. This marginal increase is due to the additional logic required to implement clock gating. However, this overhead is minimal and does not significantly impact the overall design complexity.

A significant improvement is observed in **power consumption**, where the proposed design reduces power from **1000.115 W to 62.447 W**. This drastic reduction is achieved by disabling unnecessary clock activity in idle portions of the circuit. Clock gating effectively minimizes switching activity, which is the primary source of dynamic power consumption in FPGA designs.

In terms of **delay**, the proposed design shows better performance, reducing the delay from **10.561 ns to 7.406 ns**. This improvement is due to optimized signal transitions and controlled clock distribution, which reduces unnecessary toggling and improves timing efficiency.

Overall, the proposed 32-point FFT design with clock gating demonstrates **high power efficiency and improved speed performance**, with only a negligible increase in area. Therefore, clock gating proves to be an effective technique for optimizing digital designs, especially in power-sensitive applications.

6.4 APPLICATIONS:

- **Mobile Communications (4G/5G Networks):**

FFT is a fundamental operation in many communication systems, including OFDM (Orthogonal Frequency Division Multiplexing) used in 4G and 5G technologies. The proposed method can reduce the complexity of FFT hardware implementations, leading to lower power consumption and faster processing, which is essential for real-time signal processing in mobile devices and base stations.

- **Wireless Local Area Networks (WLANs):**

In WLAN standards like Wi-Fi, FFTs are widely used for data modulation and

demodulation. The new grouping and sharing method can help optimize the FFT processor to reduce the required hardware resources, enabling more efficient data transmission in WLANs.

- **Signal Processing in IoT Devices:**

The low-complexity design of FFT processors is ideal for IoT (Internet of Things) devices, which typically have strict power, area, and latency constraints. By reducing the computational burden of FFT operations, the method can improve the overall performance of IoT systems, enabling faster data processing while preserving battery life.

- **Image and Video Processing:**

FFT plays a vital role in image and video compression, feature extraction, and enhancement. The input grouping and sharing technique can improve the speed and efficiency of these processes in embedded systems, making them more suitable for real-time applications in surveillance, video streaming, and computer vision.

- **Radar and Acoustic Signal Processing:**

FFT is widely used in radar and acoustic systems for spectrum analysis, target detection, and waveform analysis. By reducing the complexity of FFT operations, the method can contribute to lower latency and improved performance in these systems, which require fast, accurate signal processing.

- **Medical Imaging:**

In medical imaging techniques such as MRI (Magnetic Resonance Imaging) and ultrasound, FFT is employed for image reconstruction and data analysis. A low-complexity FFT implementation can enhance the processing speed and efficiency, enabling faster diagnostic results in medical applications.

- **Audio Processing:**

Audio signal processing, including tasks such as equalization, filtering, and compression, benefits from FFT. The new input grouping and sharing method can lead to more efficient FFT implementations in audio systems, enabling real-time processing for consumer electronics like smartphones, smart speakers, and hearing aids.

6.6 ADVANTAGES:

- **Reduced Computational Complexity:**

By grouping inputs and sharing common partial sums or differences, the method significantly reduces the number of multiplications and adders required in the FFT computation. This leads to lower overall computational complexity, making it more efficient compared to traditional FFT implementations.

- **Lower Hardware Resource Consumption:**

The reduction in the number of multiplications and the efficient sharing of intermediate results leads to a decrease in the hardware resources needed (such as multipliers and

adders). This is especially beneficial for resource-constrained systems like embedded devices, mobile platforms, and IoT devices.

- **Improved Power Efficiency:**

With fewer operations and reduced resource usage, the power consumption of the FFT processor is minimized. This is crucial for battery-powered devices in mobile communications, IoT applications, and portable electronics where energy efficiency is a priority.

- **Enhanced Processing Speed:**

The method optimizes the FFT computation by reducing redundant operations, leading to faster processing times. This is particularly important in real-time signal processing applications such as communications, radar, and video processing.

- **Scalability:**

The proposed method can be scaled to accommodate various FFT sizes, making it suitable for both small and large-scale FFT computations. This scalability is important for applications like wireless communication systems (e.g., 4G, 5G, WLAN) where different FFT sizes are required for different signal processing tasks.

- **Better Latency Performance:**

By reducing the number of arithmetic operations, the method improves the latency of FFT computations. This is critical for time-sensitive applications such as real-time communications, live video streaming, and medical diagnostics, where low-latency processing is essential.

- **Improved Throughput:**

With the reduction of redundant operations and better resource utilization, the method enhances the throughput of the FFT processor. This is beneficial for applications requiring high data rates, such as high-speed communication systems and real-time signal processing in multimedia devices.

- **Flexibility in Implementation:**

The method allows for the efficient design of FFT processors that can be customized to meet the specific needs of different applications. This flexibility ensures that the implementation can be tailored for optimal performance in various environments, whether in high-end base stations or low-power embedded systems.

- **Cost Savings:**

The reduced hardware resource requirements and improved energy efficiency can lead to cost savings in the production and operation of devices that rely on FFT processing. This makes the method particularly attractive for cost-sensitive applications in consumer electronics, telecommunications, and industrial systems.

Chapter 7

CONCLUSION AND FUTURE SCOPE

CONCLUSION:

The Fast Fourier Transform (FFT) plays a crucial role in digital signal processing and communication systems, but traditional implementations can be resource-intensive. This brief presents a novel low-power, low-complexity FFT architecture that leverages an innovative input grouping method to minimize multiplications involving inputs and FFT twiddle factors. Additionally, a partial sum sharing scheme is introduced to efficiently reuse hardware resources, significantly lowering adder costs. Logic synthesis results on ASIC demonstrate the effectiveness of the proposed 32-point FFT architecture, showcasing reduced hardware complexity and power consumption while maintaining performance. The approach offers significant advantages for applications in communication systems, embedded devices, and other resource-constrained environments.

FUTURE SCOPE:

The current project focuses on the design of a low-complexity FFT architecture at the hardware level, and in future, this work can be extended to develop real-time application-level systems by integrating the proposed FFT architecture into practical applications such as wireless communication, signal processing, FPGA-based embedded systems, IoT devices, and multimedia processing systems.

REFERENCES:

- [1] W. Hong et al., “Multibeam antenna technologies for 5G wireless communications,” *IEEE Trans. Antennas Propag.*, vol. 65, no. 12, pp. 6231–6249, Dec. 2017.
- [2] P. K. Daram and C.-I. H. Chen, “Smart FFT measurement for reconfigurable sensor using a wideband digital receiver,” in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, Aug. 2021, pp. 200–203.
- [3] J. Chen, J. Hu, S. Lee, and G. E. Sobelman, “Hardware efficient mixed radix-25/16/9 FFT for LTE systems,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 2, pp. 221–229, Feb. 2015.
- [4] N. Lei and X. Gu, “FFT-OT: A fast algorithm for optimal transportation,” in *Proc. IEEE/CVF ICCV*, Oct. 2021, pp. 6260–6269.
- [5] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Math. Comput.*, vol. 19, pp. 297–301, Apr. 1965.
- [6] J. Wang, S. Li, and X. Li, “Scheduling of data access for the radix-2k FFT processor using single-port memory,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1676–1689, Jul. 2020.
- [7] J. Hazarika, M. T. Khan, and S. R. Ahamed, “Low-complexity continuous-flow memory-based FFT architectures for real-valued signals,” in *Proc. 32nd Int. Conf. VLSI Des. 18th Int. Conf. Embedded Syst. (VLSID)*, Jan. 2019, pp. 46–51.
- [8] S. Liu and D. Liu, “A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 3, pp. 511–523, Mar. 2019.
- [9] Shouman Wang and Jiajia Chen, “A New Input Grouping and Sharing Method to Design Low Complexity FFT Implementation,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 2, Feb. 2024.
- [10] N. Nadar, M. Mehta, K. Bhat, J. Mendes, and R. Chaudhari, “Hardware implementation of pipelined FFT using polyphase decomposition,” in *Proc. Int. Conf. Recent Trends Electron. Inf. Commun. Technol.*, Aug. 2021, pp. 339–343.
- [11] M. Parfieniuk, “Lifting-based alternatives to the FFT for computing the 4-, 8-, and 16-point discrete Fourier transforms,” in *Proc. 29th Eur. Signal Process. Conf.*, Aug. 2021, pp. 2184–2188.
- [12] IEEE Standard for Information Technology–Telecommunications and Information Exchange Between Systems–Local and Metropolitan Area Networks–Specific Requirements–Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications

Amendment 3: Enhancements for Very High Throughput in the 60 GHz Band, IEEE Standard 802.11ad-2012, Dec. 2012.

- [13] B. Liu et al., “Precision adaptive MFCC based on R2SDF-FFT and approximate computing for low-power speech keywords recognition,” *IEEE Circuits Syst. Mag.*, vol. 21, no. 4, pp. 24–39, 4th Quart., 2021.
- [14] H.-Y. Chin, P.-Y. Tsai, and S.-Y. Lee, “Implementation of a twodimensional FFT/IFFT processor for real-time high-resolution synthetic aperture radar imaging,” in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2021, pp. 211–216.
- [15] S.-C. Hsu, S.-J. Huang, S.-G. Chen, S.-C. Lin, and M. Garrido, “A 128-point multi-pathSC FFT architecture,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Oct. 2020, pp. 1–5.
- [16] M. Garrido and P. Paz, “Optimum MDC FFT hardware architectures in terms of delays and multiplexers,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 3, pp. 1003–1007, Mar. 2021.
- [17] X. Han, J. Chen, and S. Rahardja, “A new twiddle factor merging method for low complexity and high speed FFT architecture,” in *Proc. IEEE Int. Circuits Syst. Symp.*, Sep. 2019, pp. 1–5.
- [18] X. Han, J. Chen, B. Qin, and S. Rahardja, “A novel area-power efficient design for approximated small-point FFT architecture,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4816–4827, Dec. 2020.
- [19] T.-W. Siu, C.-W. Sham, and F. C. M. Lau, “Operating frequency improvement on FPGA implementation of a pipeline large-FFT processor,” in *Proc. 19th ICACT*, Feb. 2017, pp. 5–9.
- [20] T.-T. Hoang, D.-H. Le, and C.-K. Pham, “VLSI design of floating-point twiddle factor using adaptive CORDIC on various iteration limitations,” in *Proc. IEEE 12th Int. Symp. Embedded Multicore/Many-Core Syst. Chip*, Sep. 2018, pp. 225–232.
- [21] W. M. Gentleman and G. Sande, “Fast Fourier transforms: For fun and profit,” in *Proc. AFIPS*, San Francisco, CA, USA, Nov. 1966, pp. 563–578.
- [22] J. Ding, J. Chen, and C.-H. Chang, “A new paradigm of common subexpression elimination by unification of addition and subtraction,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1605–1617, Oct. 2016.
- [23] M. Garrido and P. Malagón, “The constant multiplier FFT,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 322–335, Jan. 2021.

Classification of Project

Classification of Project	Application	Product	Research	Review
			✓	

Note: Tick Appropriate category.

PROJECT COs– POs/PSOs MAPPING

Course Title: A New Input Grouping and Sharing Method to Design Low Complexity 32-Point FFT Implementation,

Course Code: EC3572

Department: ELECTRONICS AND COMMUNICATION ENGINEERING

Batch Number: D12

Academic Year: 2025-26

1. Project Course Outcomes (COs)

CO No	Course Outcome Statement
CO1	Identify, formulate, and analyze complex real-world problems by applying fundamental principles of Electronics and Communication Engineering.
CO2	Design and develop electronic or communication systems, components, or processes that meet specified requirements while considering realistic constraints such as economic, environmental, societal, and safety aspects.
CO3	Select and apply appropriate modern engineering tools, software, and hardware platforms to model, simulate, implement and validate solutions in Electronics and Communication Engineering.
CO4	Function effectively as an individual and as a member of a multidisciplinary team, demonstrating project management skills and adherence to timelines.
CO5	Communicate effectively through technical reports, documentation, and presentations, while adhering to professional ethics and recognizing the impact of engineering solutions on society.

2. Program Outcomes (POs)

PO No	Program Outcome Statement
PO1	Engineering Knowledge
PO2	Problem Analysis
PO3	Design / Development of Solutions
PO4	Conduct Investigations of Complex Problems
PO5	Engineering Tool Usage
PO6	The Engineer and the World
PO7	Ethics
PO8	Individual and Collaborative Team Work
PO9	Communication
PO10	Project Management and Finance
PO11	Life Long Learning

3. Program Specific Outcomes (PSOs)

PSO No	PSO Statement
PSO1	Designing electronics and communication systems in the domains of VLSI, embedded systems, signal processing and RF communications, and applying modern tools.
PSO2	Applying the contextual knowledge of Electronics and Communication Engineering to design, develop, analyze and test systems containing hardware and software components taking into societal, environmental, health, safety, legal, cultural, ethical and economical considerations.

4. CO – PO / PSO Mapping Table (3–High, 2–Moderate, 1–Low)

CO \ PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	3	2	2	1	–	–	–	–	–	1	3	3
CO2	2	2	3	2	2	2	1	–	–	1	–	3	3
CO3	2	1	2	2	3	–	–	–	–	–	1	2	3
CO4	–	–	–	–	–	–	1	3	2	3	–	-	1
CO5	–	–	–	–	–	2	3	2	3	1	2	1	1

Justification of CO-PO/PSO Mapping

The CO–PO mapping for the capstone project course is designed to ensure comprehensive attainment of all program outcomes through integrated learning experiences.

- CO1 emphasizes engineering knowledge and problem analysis, thereby strongly supporting PO1 and PO2, while also contributing to investigation and preliminary design aspects (PO3, PO4).
- CO2 focuses on system design under realistic constraints, aligning closely with PO3 and moderately addressing societal considerations (PO6) and ethical awareness (PO7).
- CO3 highlights the use of modern engineering tools, directly mapping to PO5 and supporting analytical and investigative skills.
- CO4 develops teamwork, leadership, and project execution abilities, strongly contributing to PO8 and PO10, along with communication skills (PO9).
- CO5 ensures effective communication, ethical responsibility, and awareness of global and societal impacts, thereby addressing PO7, PO9, and PO6, while also encouraging lifelong learning (PO11).

5. Sustainable Development Goals (SDG) Alignment

SDG No	Goal	Relevance to the Project
SDG 9	Industry, Innovation, and Infrastructure	The project develops an optimized FFT architecture for efficient digital systems
SDG 7	Affordable and Clean Energy	Clock gating reduces power consumption, improving energy efficiency
SDG 11	Sustainable Cities and Communities	Efficient signal processing supports smart communication systems
SDG 12	Responsible Consumption and Production	Reduced hardware complexity minimizes resource usage

Signature(s) of Students:

Signature of Project Supervisor:

- 1.
- 2.
- 3
- 4.

APPENDIX

Verilog codes for With Clock Gating

Modules:

1. Mux_2to1
2. UAS
3. Top_16
4. Flipflop_based_clk
5. Top_32

Test Bench Code:

1. Top_tb

1. Mux_2to1

```
`timescale 1ns / 1ps
module mux_2to1(
    input signed[15:0]in0,
    input signed[15:0]in1,
    input signed[15:0]sel,
    output signed[15:0]out
);
    assign out = sel ? in1 : in0;
endmodule
```

2. UAS

```
`timescale 1ns / 1ps
module UAS(
    input signed [15:0]x,cin,
    input signed [15:0]y,bin,
    output signed [15:0] s,cout,
    output signed [15:0] d,dout
);
    Wire signed [15:0] o;
    assign o=x^y;
    mux_2to1 m1(y,cin,o,cout);
    //assign cout = o ? y:cin;
    assign s=cin^o;
    assign d=o^bin;
    //assign dout = 0?y:bin;
    mux_2to1 m2(bin,y,o,dout);
```

endmodule

3. Top_16

```
`timescale 1ns / 1ps
module top_16( input clk,
    input signed [15:0] A0_real, A0_imag,
    input signed [15:0] A1_real, A1_imag,
    input signed [15:0] A2_real, A2_imag,
    input signed [15:0] A3_real, A3_imag,
    input signed [15:0] A4_real, A4_imag,
    input signed [15:0] A5_real, A5_imag,
    input signed [15:0] A6_real, A6_imag,
    input signed [15:0] A7_real, A7_imag,
    input signed [15:0] A8_real, A8_imag,
    input signed [15:0] A9_real, A9_imag,
    input signed [15:0] A10_real, A10_imag,
    input signed [15:0] A11_real, A11_imag,
    input signed [15:0] A12_real, A12_imag,
    input signed [15:0] A13_real, A13_imag,
    input signed [15:0] A14_real, A14_imag,
    input signed [15:0] A15_real, A15_imag,

    output reg signed [15:0] X0_real, X0_imag,
    output reg signed [15:0] X1_real, X1_imag,
    output reg signed [15:0] X2_real, X2_imag,
    output reg signed [15:0] X3_real, X3_imag,
    output reg signed [15:0] X4_real, X4_imag,
    output reg signed [15:0] X5_real, X5_imag,
    output reg signed [15:0] X6_real, X6_imag,
    output reg signed [15:0] X7_real, X7_imag,
    output reg signed [15:0] X8_real, X8_imag,
    output reg signed [15:0] X9_real, X9_imag,
    output reg signed [15:0] X10_real, X10_imag,
    output reg signed [15:0] X11_real, X11_imag,
    output reg signed [15:0] X12_real, X12_imag,
    output reg signed [15:0] X13_real, X13_imag,
    output reg signed [15:0] X14_real, X14_imag,
    output reg signed [15:0] X15_real, X15_imag

);
wire signed [15:0] b0_real, b0_imag,
    b1_real, b1_imag,
    b2_real, b2_imag,
    b3_real, b3_imag,
    b4_real, b4_imag,
    b5_real, b5_imag,
    b6_real, b6_imag,
    b7_real, b7_imag,
    b8_real, b8_imag,
```



```

b9_real, b9_imag,
b10_real, b10_imag,
b11_real, b11_imag,
b12_real, b12_imag,
b13_real, b13_imag,
b14_real, b14_imag,
b15_real, b15_imag;

UAS b1(A0_real, A0_imag, A1_real, A1_imag, b0_real, b0_imag, b1_real, b1_imag); //1
UAS b2(A2_real, A2_imag, A3_real, A3_imag, b2_real, b2_imag, b3_real, b3_imag); //2
UAS b3(A4_real, A4_imag, A5_real, A5_imag, b4_real, b4_imag, b5_real, b5_imag); //3
UAS b4(A6_real, A6_imag, A7_real, A7_imag, b6_real, b6_imag, b7_real, b7_imag); //4
UAS b5(A8_real, A8_imag, A9_real, A9_imag, b8_real, b8_imag, b9_real, b9_imag); //5
UAS b6(A10_real, A10_imag, A11_real, A11_imag, b10_real, b10_imag, b11_real,
b11_imag); //6
UAS b7(A12_real, A12_imag, A13_real, A13_imag, b12_real, b12_imag, b13_real,
b13_imag); //7
UAS b8(A14_real, A14_imag, A15_real, A15_imag, b14_real, b14_imag, b15_real,
b15_imag); //8

///stage2
wire signed [15:0] c0_real, c0_imag,
c1_real, c1_imag,
c2_real, c2_imag,
c3_real, c3_imag,
c4_real, c4_imag,
c5_real, c5_imag,
c6_real, c6_imag,
c7_real, c7_imag,
c8_real, c8_imag,
c9_real, c9_imag,
c10_real, c10_imag,
c11_real, c11_imag,
c12_real, c12_imag,
c13_real, c13_imag;

UAS ab1(b0_real, b0_imag, b2_real, b2_imag, c0_real, c0_imag, c1_real, c1_imag); //1
UAS ab2(b4_real, b4_imag, b6_real, b6_imag, c2_real, c2_imag, c3_real, c3_imag); //2
UAS ab3(b5_real, b5_imag, b7_real, b7_imag, c4_real, c4_imag, c5_real, c5_imag); //3
UAS ab4(b8_real, b8_imag, b10_real, b10_imag, c6_real, c6_imag, c7_real, c7_imag); //4

UAS ab5(b9_real, b9_imag, b11_real, b11_imag, c8_real, c8_imag, c9_real, c9_imag); //5
UAS ab6(b12_real, b12_imag, b14_real, b14_imag, c10_real, c10_imag, c11_real, c11_imag);
//6
UAS ab7(b13_real, b13_imag, b15_real, b15_imag, c12_real, c12_imag, c13_real, c13_imag);
//7

```

//// stage 3

wire signed [15:0] r1,r2,r3,r4,r5,r6,r7,r8,r9,r10,r11,r12,r13,r14;
wire signed [15:0] i1,i2,i3,i4,i5,i6,i7,i8,i9,i10,i11,i12,i13,i14;
wire signed [15:0] s1,s2,s3;

```

assign s1=0.9238;
assign s2=0.707;
assign s3=0.382;
/// for real
assign r1 = c0_real + c2_real; //1
assign r2 = c0_real - c2_real; //2
assign r3 = (s2+b3_real) + c3_real; //3
assign r4 = (s2+b3_real) - c3_real; //4
assign r5 = (s2+b1_real) + c5_real; //5
assign r6 = (s2+b1_real) - c5_real; //6
assign r7 = c6_real + c10_real; //7
assign r8 = s2+c6_real - s2+c10_real; //8
assign r9 = s1+c7_real + s3+c11_real; //9
assign r10 = s3+c7_real - s1+c11_real; //10
assign r11 = s2+c8_real + s2+c12_real; //11
assign r12 = c8_real - c12_real; //12
assign r13 = s3+c9_real + s1+c13_real; //13
assign r14 = s1+c9_real - s3+c13_real; //14

// for imag
assign i1 = c0_imag + c2_imag; //1
assign i2 = c0_imag - c2_imag; //2
assign i3 = (s2+b3_imag) + c3_imag; //3
assign i4 = (s2+b3_imag) - c3_imag; //4
assign i5 = (s2+b1_imag) + c5_imag; //5
assign i6 = (s2+b1_imag) - c5_imag; //6
assign i7 = c6_imag + c10_imag; //7
assign i8 = s2+c6_imag - s2+c10_imag; //8
assign i9 = s1+c7_imag + s3+c11_imag; //9
assign i10 = s3+c7_imag - s1+c11_imag; //10
assign i11 = s2+c8_imag + s2+c12_imag; //11
assign i12 = c8_imag - c12_imag; //12
assign i13 = s3+c9_imag + s1+c13_imag; //13
assign i14 = s1+c9_imag - s3+c13_imag; //14

```

```

/// stage 4
wire signed [15:0] re1,re2,re3,re4,re5,re6,re7,re8,re9,re10,re11,re12,re13,re14;
wire signed [15:0] im1,im2,im3,im4,im5,im6,im7,im8,im9,im10,im11,im12,im13,im14;

/// for real
assign re1 = c1_real - r8;          //1
assign re2 = r1+r9;                 //2
assign re3 = r3;                    //3
assign re4 = r4+r10;                //4
assign re5 = c4_real + r11;         //5
assign re6 = r5 - r13;              //6
assign re7 = r6 - r14;              //7
assign re8 = r1 + r7;               //8
assign re9 = c1_real + r8;          //9
assign re10 = r3 - r9;              //10
assign re11 = r4 - r10;             //11
assign re12 = c4_real + r11;        //12
assign re13 = r5 + r13;             //13
assign re14 = r6 - r14;            //14

// for imag
assign im1 = c1_imag - i8;          //1
assign im2 = i1+i9;                 //2
assign im3 = i3;                    //3
assign im4 = i4+i10;                //4
assign im5 = c4_imag + i11;         //5
assign im6 = i5 - i13;              //6
assign im7 = i6 - i14;              //7
assign im8 = i1 + i7;               //8
assign im9 = c1_imag + i8;          //9
assign im10 = i3 - i9;              //10
assign im11 = i4 - i10;             //11
assign im12 = c4_imag + i11;        //12
assign im13 = i5 + i13;             //13
assign im14 = i6 - i14;            //14

//// final stage

//// for real

always @(posedge clk) begin

    X0_real <= re1 ;
    X1_real <= re2 ;
    X2_real <= re3 ;
    X3_real <= re4 ;
    X4_real <= re5 ;
    X5_real <= re6 ;
    X6_real <= re6 ;

```

```

X7_real  <= re7 ;
X8_real  <= re8 ;
X9_real  <= re9 ;
X10_real <= re10;
X11_real <= re11;
X12_real <= re12;
X13_real <= r12 ;
X14_real <= re13;
X15_real <= re14;

/// for image

X0_imag  <= im1 ;
X1_imag  <= im2 ;
X2_imag  <= i2  ;
X3_imag  <= im3 ;
X4_imag  <= im4 ;
X5_imag  <= im5 ;
X6_imag  <= im6 ;
X7_imag  <= im7 ;
X8_imag  <= im8 ;
X9_imag  <= im9 ;
X10_imag <= im10;
X11_imag <= im11;
X12_imag <= im12;
X13_imag <= i12 ;
X14_imag <= im13;
X15_imag <= im14;

end
endmodule

```

4. Flipflop_based_clk

```

module flipflop_based_clk(q_out,clk,gated_clk);
input q_out;
input clk;
output gated_clk;
wire reset,enable;
D_FlipFlop d0(clk,reset,enable,q_out);
and (gated_clk,q_out,clk);
endmodule

module D_FlipFlop (clk,reset,enable,q_out);
input clk;
input reset;
input enable;
output reg q_out;
always @(posedge clk or posedge reset)

```

```
begin
    if (reset)
        q_out<= 1'b0;
    else
        q_out<= enable;
    end
endmodule
```

5. TOP_32

```
`timescale 1ns / 1ps
module top_32(
    input clk,en,
    input signed [15:0] A0_real, A0_imag,
    input signed [15:0] A1_real, A1_imag,
    input signed [15:0] A2_real, A2_imag,
    input signed [15:0] A3_real, A3_imag,
    input signed [15:0] A4_real, A4_imag,
    input signed [15:0] A5_real, A5_imag,
    input signed [15:0] A6_real, A6_imag,
    input signed [15:0] A7_real, A7_imag,
    input signed [15:0] A8_real, A8_imag,
    input signed [15:0] A9_real, A9_imag,
    input signed [15:0] A10_real, A10_imag,
    input signed [15:0] A11_real, A11_imag,
    input signed [15:0] A12_real, A12_imag,
    input signed [15:0] A13_real, A13_imag,
    input signed [15:0] A14_real, A14_imag,
    input signed [15:0] A15_real, A15_imag,
    input signed [15:0] A16_real, A16_imag,
    input signed [15:0] A17_real, A17_imag,
    input signed [15:0] A18_real, A18_imag,
    input signed [15:0] A19_real, A19_imag,
    input signed [15:0] A20_real, A20_imag,
    input signed [15:0] A21_real, A21_imag,
    input signed [15:0] A22_real, A22_imag,
    input signed [15:0] A23_real, A23_imag,
    input signed [15:0] A24_real, A24_imag,
    input signed [15:0] A25_real, A25_imag,
    input signed [15:0] A26_real, A26_imag,
    input signed [15:0] A27_real, A27_imag,
    input signed [15:0] A28_real, A28_imag,
    input signed [15:0] A29_real, A29_imag,
    input signed [15:0] A30_real, A30_imag,
    input signed [15:0] A31_real, A31_imag,

    output signed [15:0] X0_real, X0_imag,
    output signed [15:0] X1_real, X1_imag,
    output signed [15:0] X2_real, X2_imag,
```

```

    output signed [15:0] X3_real, X3_imag,
    output signed [15:0] X4_real, X4_imag,
    output signed [15:0] X5_real, X5_imag,
    output signed [15:0] X6_real, X6_imag,
    output signed [15:0] X7_real, X7_imag,
    output signed [15:0] X8_real, X8_imag,
    output signed [15:0] X9_real, X9_imag,
    output signed [15:0] X10_real, X10_imag,
    output signed [15:0] X11_real, X11_imag,
    output signed [15:0] X12_real, X12_imag,
    output signed [15:0] X13_real, X13_imag,
    output signed [15:0] X14_real, X14_imag,
    output signed [15:0] X15_real, X15_imag,
    output signed [15:0] X16_real, X16_imag,
    output signed [15:0] X17_real, X17_imag,
    output signed [15:0] X18_real, X18_imag,
    output signed [15:0] X19_real, X19_imag,
    output signed [15:0] X20_real, X20_imag,
    output signed [15:0] X21_real, X21_imag,
    output signed [15:0] X22_real, X22_imag,
    output signed [15:0] X23_real, X23_imag,
    output signed [15:0] X24_real, X24_imag,
    output signed [15:0] X25_real, X25_imag,
    output signed [15:0] X26_real, X26_imag,
    output signed [15:0] X27_real, X27_imag,
    output signed [15:0] X28_real, X28_imag,
    output signed [15:0] X29_real, X29_imag,
    output signed [15:0] X30_real, X30_imag,
    output signed [15:0] X31_real, X31_imag
);
wire gated_clk;
flipflop_based_clk f1 (en,clk,gated_clk);
top_16 u1(gated_clk,
A0_real, A0_imag,
A1_real, A1_imag,
A2_real, A2_imag,
A3_real, A3_imag,
A4_real, A4_imag,
A5_real, A5_imag,
A6_real, A6_imag,
A7_real, A7_imag,
A8_real, A8_imag,
A9_real, A9_imag,
A10_real, A10_imag,
A11_real, A11_imag,
A12_real, A12_imag,
A13_real, A13_imag,
A14_real, A14_imag,
A15_real, A15_imag,
X0_real, X0_imag,

```

```
X1_real, X1_imag,
X2_real, X2_imag,
X3_real, X3_imag,
X4_real, X4_imag,
X5_real, X5_imag,
X6_real, X6_imag,
X7_real, X7_imag,
X8_real, X8_imag,
X9_real, X9_imag,
X10_real, X10_imag,
X11_real, X11_imag,
X12_real, X12_imag,
X13_real, X13_imag,
X14_real, X14_imag,
X15_real, X15_imag );
```

```
top_16 u2(gated_clk,
A16_real, A16_imag,
A17_real, A17_imag,
A18_real, A18_imag,
A19_real, A19_imag,
A20_real, A20_imag,
A21_real, A21_imag,
A22_real, A22_imag,
A23_real, A23_imag,
A24_real, A24_imag,
A25_real, A25_imag,
A26_real, A26_imag,
A27_real, A27_imag,
A28_real, A28_imag,
A29_real, A29_imag,
A30_real, A30_imag,
A31_real, A31_imag,
```

```
X16_real, X16_imag,
X17_real, X17_imag,
X18_real, X18_imag,
X19_real, X19_imag,
X20_real, X20_imag,
X21_real, X21_imag,
X22_real, X22_imag,
X23_real, X23_imag,
X24_real, X24_imag,
X25_real, X25_imag,
X26_real, X26_imag,
X27_real, X27_imag,
X28_real, X28_imag,
X29_real, X29_imag,
X30_real, X30_imag,
X31_real, X31_imag );
endmodule
```

Status of Project Publication

The research paper based on this project has been successfully submitted to i-Manager Publications and is currently under the review stage.



Dashboard Tickets Documents Library News

3

Vaddimukkala Venu

Author



SUBMITTED PAPERS
1

UNDER REVIEW
1

WITH EIC
0

PRE PRESS
0

PUBLISHED
0

New Submission

Papers

Search papers...



S.No	Article	Article Title	Journal	Status	Action
1	JELE0028	A Low Complexity 32-Point FFT Architecture Using Input Grouping, Partial Sum Sharing, and Clock Gating Technique	i-manager's Journal on Electronics Engineering	Under Review	

Showing 1-1 of 1 entries

Rows per page101-1 of 1

Recent Events

The paper JELE0028 has been sent for reviewer assignment

2026-04-01 12:04:13

Plagiarism for the paper JELE0028 has been checked

2026-03-23 15:12:16

The paper JELE0028 has been sent to Associate Editor

2026-03-23 15:12:16

The paper JELE0028 has been submitted

2026-03-23 10:45:47

A Low Complexity 32-Point FFT Architecture Using Input Grouping, Partial Sum Sharing, and Clock Gating Technique

Dr. V. V. K. D. V. Prasad¹, Vaddimukkala Venu², Saniya Begum³, Yasam Gopalakrishna⁴, Rokkam Varshitha⁵

¹Professor & Convener (IQAC), Department of Electronics and Communication Engineering,
Seshadri Rao Gudlavalleru Engineering College, Gudlavalleru, Andhra Pradesh, India.

Email: varrevkdvp@rediffmail.com

²³⁴⁵UG Students, Department of Electronics and Communication Engineering,
Seshadri Rao Gudlavalleru Engineering College, Gudlavalleru, Andhra Pradesh, India.

² venuvaddimukkala9@gmail.com, ³ saniyabegum79814@gmail.com, ⁴ yasamgopalakrishna@gmail.com, ⁵ rokkamvarshitha@gmail.com

Abstract: The Fast Fourier Transform (FFT) is a fundamental algorithm widely used in digital signal processing for efficient frequency-domain analysis. However, conventional FFT architectures suffer from high computational complexity, increased power consumption, and large hardware requirements. This paper proposes a low-complexity 32-point FFT architecture based on an innovative input grouping and partial sum sharing technique. The proposed method reduces redundant arithmetic operations by grouping inputs with similar twiddle factors and reusing intermediate results. Additionally, a flip-flop-based clock gating technique is incorporated to minimize dynamic power consumption by disabling inactive clock regions. The architecture is implemented using Verilog HDL and validated using Xilinx Vivado. Experimental results demonstrate a significant reduction in power consumption and delay with only a marginal increase in area. The proposed design is suitable for real-time and low-power applications such as wireless communication, IoT, and embedded systems.

Keywords: FFT, Input Grouping, Partial Sum Sharing, Clock Gating, Low Power, FPGA

1. Introduction

In modern digital signal processing systems, the Fast Fourier Transform (FFT) plays a vital role in transforming time-domain signals into frequency-domain representations. This transformation is essential in applications such as orthogonal frequency division multiplexing (OFDM), image enhancement, spectrum analysis, and biomedical signal processing. The FFT reduces the computational burden of the Discrete Fourier Transform (DFT) from quadratic complexity $O(N^2)$ to logarithmic complexity $O(N \log N)$, making it suitable for real-time processing. However, despite this reduction, practical hardware implementations of FFT still involve a large number of multipliers, adders, and memory elements. These components contribute significantly to power consumption and silicon area, especially in embedded and portable devices.

With the advent of portable and battery-operated devices, the need for a low power and high performance FFT processor is on the rise. Conventional FFT structures perform redundant operations and work under full clock activity even if some of the blocks are idle. This results in unnecessary power dissipation. There is a need to reduce the arithmetic complexity, minimize redundant operations, and reduce the dynamic power. The conventional FFT architectures are facing many challenges like a large number of complex multiplications, redundant operations, continuous switching of the clocks leading to the loss of dynamic power, and delay due to poor data flow.

The objective of this work is to design an efficient FFT architecture that reduces computational complexity using input grouping, minimizes hardware usage using partial sum sharing, reduces

power consumption using clock gating technique and achieves better delay performance. The key contributions of this paper are:

1. A novel input grouping mechanism to reduce multipliers
2. Efficient partial sum reuse strategy
3. Integration of flip-flop-based clock gating
4. FPGA-based implementation and performance evaluation
5. Comparative analysis demonstrating improved efficiency

2. Literature Review

The N-point DFT $X(k)$ of $x(n)$ is described using equation (1).

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{nk}, k = 0 \text{ to } N - 1 \quad (1)$$

Where $x(n)$ is the discrete time is input sequence and $X(k)$ is its frequency domain representation. $W_N^{nk} = e^{-j2\pi nk/N}$ is known as the twiddle factor.

Computing DFT directly using equation (1) involves N^2 multiplications and $N(N - 1)$ additions, which are computationally intensive operations, especially for large N. Hence, FFT was proposed to efficiently compute DFT. Cooley-Tukey radix-2 decimation in time (DIT) FFT is the simplest and most popular algorithm and is defined as:

$$X(k) = X_1(k) + W_N^k X_2(k) \quad (2)$$

$$X_1(k) = \sum_{n=0}^{\frac{N}{2}-1} x(2n)W_N^{nk} \quad (3)$$

$$X_2(k) = \sum_{n=0}^{\frac{N}{2}-1} x(2n + 1)W_N^{nk} \quad (4)$$

By recursively using equation (2), the DFT of an N-point signal can be broken down into $\log_2 N$ stages, and in each stage, there are N/2 two-point DFTs. The computation scheme of a 16-point decimation-in-time Fast Fourier Transform (DIT-FFT) in four stages is demonstrated in Figure 1.

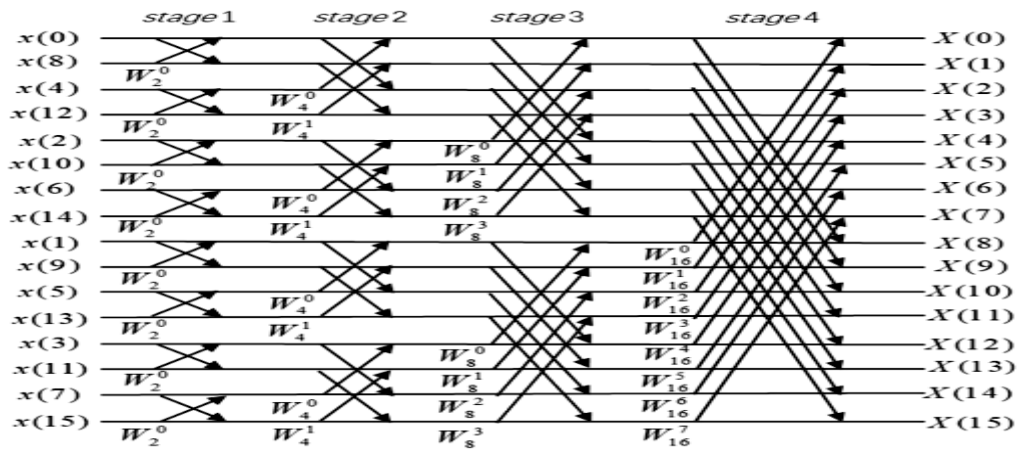


Figure 1: Implementation of 16-point Decimation in Time FFT algorithm

FFT was mostly implemented in software. However, with the introduction of system-on-a-chip (SoC) and other techniques, the implementation of FFT in hardware becomes possible in order to reduce the area and power consumption under the circumstances of maintaining the output throughout the process. The existing architectures in the implementation of FFT are mostly categorized into two types: memory-based architectures [6]-[8] and pipelined architectures [9]-[11]. The first type of architecture utilizes a single butterfly processing element (PE) and memory blocks in the implementation of the FFT algorithm. The utilization of the computing resources is reduced in this architecture, but the data throughput is low in this architecture. For applications that require higher throughputs, such as IEEE 802.11ad [12], the pipelined architectures are mostly used in the implementation of the FFT algorithm. These pipelined architectures can be further classified into single-path delay feedback (SDF) [13], [14], multi-path delay feedback (MDF) [15] and multipath delay commutator (MDC) [16].

Because of the need to add more butterfly PEs and FFT rotators to execute twiddle factors' multiplication, the hardware cost of the pipelined architectures is substantially increased. To overcome this problem, CSD arithmetic and common sub-expression sharing techniques have been proposed. In order to reduce the number of adders needed in the hardware implementation of the FFT architecture, the authors of [17] have proposed an efficient parallel FFT architecture that can be achieved by truncating the insignificant bits. A new approximate parallel FFT architecture has been proposed recently in [18] where a new twiddle factors merging technique is presented to execute the merged expressions. Although the low area-power design is achieved in [18], more efficient hardware can be achieved within the margin of error.

3. Proposed Method

In this paper, a new 32-point Fast Fourier Transform (FFT) architecture is introduced, which is based on a 16-point FFT as the fundamental computing component and building block. Instead of designing a 32-point FFT, which involves a larger number of arithmetic operations, hardware resources, and power consumption, the design is broken down into two 16-point FFT operations, making the design much easier and reducing the number of complex multiplications and additions. This not only reduces the complexity of the design but also aids the efficient use of hardware resources like adders, multipliers, and memory. Furthermore, since the 16-point FFT module has already been optimized using techniques like input grouping and partial sum sharing, the 32-point FFT module inherits the advantages of the 16-point FFT module, thus improving the performance. However, it is observed that the power consumption of the circuit increases due to the continuous switching of the clocks. This problem can be solved by implementing a clock gating technique.

3.1 Flip flop-based clock gating technique

In various applications, latch-based designs are converted to flip-flop-based designs. The flip-flop is divided, creating two latches from the master-slave flip-flop. This results in a D flip-flop with an AND gate. The flip-flop-based clock gating technique is used to minimize dynamic power consumption in digital systems by gating the clock during idle periods in the system. This technique differs from latch-based clock gating, as it provides a glitch-free gated clock through synchronization of the enable signal (En) with the clock edge, as achieved through flip-flop-based clock gating. This technique samples the enable signal using a D flip-flop on the rising edge of the clock signal. The output of the flip-flop is then performed logical AND operation with the clock signal to generate a gated clock (CLK_GATED) as shown in figure 2.

This ensures that the gated clock transitions only occur at well-defined clock edges, preventing glitches and ensuring reliable operation.

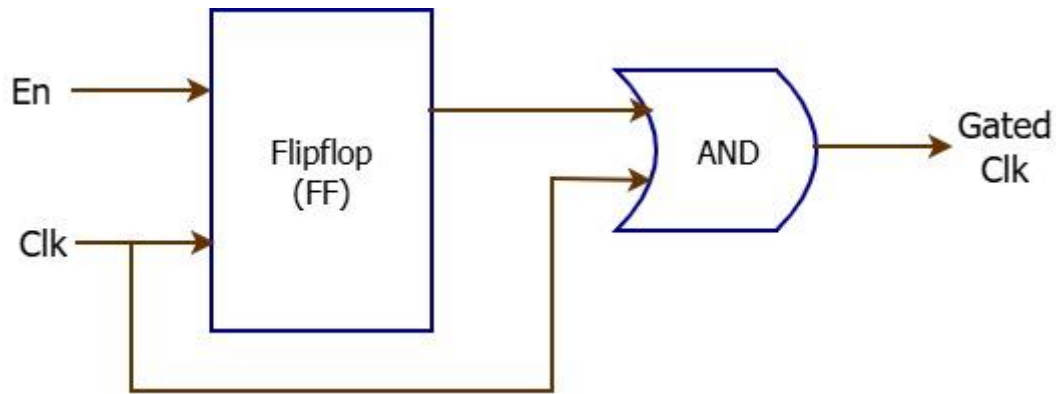


Figure 2: Flip flop-based clock gating technique

As depicted in Table 1, the gated clock will be driven to a high level if the output of the flip-flop and the clock signal are both high. Otherwise, the gated clock will stay in the zero state. It can be concluded that, if the clock signal is driven to the sleep mode, the gated clock will stay in the zero state. This method can be used to prevent the enable signal from being sampled on the falling edge of the clock signal. This method is commonly applied to low-power processors, digital signal processing circuits, and FPGA/ASIC designs. Compared to the latch-based method of clock gating, the flip-flop method provides better timing closure, solves the problem of race conditions, and can be easily incorporated into a synchronous design.

Table 1: Truth Table of Clock Gating

Clock (Clk)	Enable(En)	FF output	Gated Clk	Operation
0	0	0	0	Clk OFF
1	0	0	0	Clk OFF
0	1	1	0	Waiting
1	1	1	1	Clk ON

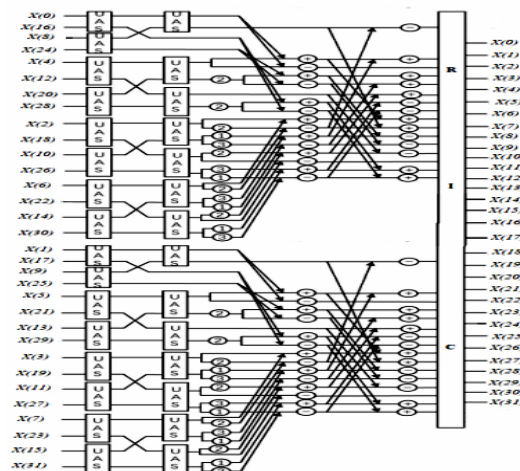


Figure 3: Proposed 32-point FFT architecture

The proposed 32-point FFT architecture using clock gating logic of figure 2 is shown in figure 3. The architecture is divided into three main parts:

1. *Left Side (Input Section)*: Inputs are labeled as: $X(0), X(1), X(2), \dots, X(31)$. These inputs are arranged in groups (not processed individually like traditional FFT)

2. *Middle Section (Computation Core)* : The unit ADD or SUBTRACT block (UAS) acts like simplified butterfly units, which are used in the design to combine input pairs and reduce number of operations before multiplication. After UAS operations, intermediate results are reused multiple times to avoid re-computing the same values and extra adders and multipliers. The criss-cross lines in the middle indicate data routing between stages and proper alignment of FFT outputs, thus replacing complex memory-based data shuffling and traditional stage-wise FFT connections.

3. *Right Side (Output Section)*: Final FFT outputs are produced from this stage, which are separated into real and imaginary parts.

4. Results and Discussion

4.1 Implementation of Proposed Architecture using Verilog HDL

The 32-point Fast Fourier Transform (FFT) architecture and the flip-flop-based clock gating technique are implemented using the Verilog HDL programming language. The architecture is based on a modular approach, in which each component is designed as a separate module and then combined to form the entire system. The top-level module, named `top.v`, is the main controller and connects all the modules together and controls the flow of data in the entire system. The module controls the input signals, clock, and enables signals required for the entire system. The 32-point FFT module is designed as a separate module named `top_32.v` and is based on a divide-and-conquer approach using two 16-point FFT blocks. The input data is split into even and odd values and is then processed independently in the 16-point FFT block.

The "Unit Arithmetic Structure (UAS.v)" module is used for arithmetic operations such as addition and subtraction that are needed for butterfly computation. The module is used for both real and imaginary parts of the Fast Fourier Transform (FFT). Therefore, it is possible to handle complex numbers. The "multiplexor (mux_2to1.v)" module is used to control the selection of data paths of input information at different stages of computation. To make the design more power-efficient, a "flip-flop-based clock gating (flipflop_based_clk.v)" module is used. The clock is controlled through this module's enable signal. The clock is enabled only during computation and disabled during idle time. This reduces unnecessary switching of the clock and hence minimizes dynamic power dissipation. The design is simulated and verified. The "RTL Schematics" of the design is created to ensure that the hardware is correctly structured. The "Simulation waveforms" of the design are also analyzed to ensure that it is functioning correctly. The addition of the clock gating feature to the FFT architecture makes it more power-efficient.

The schematic of the Register Transfer Level (RTL) of the suggested 32-point Fast Fourier Transform (FFT) architecture, as designed using the Xilinx Vivado tool after the synthesis of the Verilog code, is presented in Figure 4. The schematic of the designed architecture is a high-level diagram that indicates the interconnections between the modules of the architecture,

including the 16-point FFT blocks, the Unit Arithmetic Structure (UAS), the multiplexers, and the clock gating block. The schematic indicates that the 32-point FFT architecture is designed as a modular architecture that includes two 16-point FFT modules, where the input signals are distributed among the modules and the outputs are integrally connected through the interconnections of the architecture. The flow of the data among the modules is clearly indicated in the schematic, which shows the flow of the signals among the successive computation blocks of the architecture. The clock gating block is included in the architecture, and the clock signal is controlled through the enable signal so that the clock is active only during the computation process. The schematic of the designed architecture is an assurance of the logical correctness of the designed architecture and its adherence to the suggested architecture before moving further to the implementation process.



Figure 4: RTL Schematic of Proposed 32-point FFT Architecture

4.1.1 Simulation Waveform

The simulation waveform in Figure 5 shows the functional verification of the 32-point FFT architecture without clock gating. The input signals include the real and imaginary parts of the signals (A0_real, A0_imag, ...), and the output signals include the real and imaginary parts of the signals (X15_real, X15_imag, ...). The waveform shows the stability of the signals during the simulation. The absence of clock gating causes the clock signal to be always present, thus ensuring the registers switch continuously during the simulation time. The outputs show the correctness of the design.

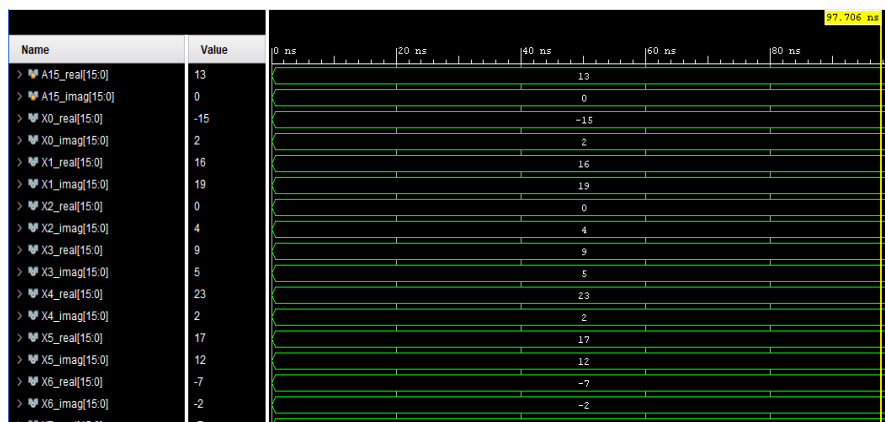


Figure 5: Simulation of 32-Point FFT Architecture Without Clock Gating

4.1.2 Area Utilization

The area report defines the hardware resources used in the design is shown in figure 6. The 32-point FFT architecture utilizes approximately 3290 Slice LUTs and 2048 bonded IOBs. The lack of clock gating implementation leads to no optimization in the registers and logic reduction, causing higher utilization in this case.

Name ¹	Slice LUTs (134600)	Bonded IOB (400)	
N top_32	3290	2048	

Figure 6: Device Utilization

4.1.3 Delay Analysis

The delay analysis report in table 2 shows the timing performance of the 32-point FFT design. The critical path delay observed is approximately 10.561 ns, which includes logic delay (~5.610 ns) and net delay (~4.951 ns). The slack value is positive, indicating that the design meets timing requirements. However, due to the absence of clock gating, unnecessary switching contributes indirectly to timing inefficiencies in larger designs.

Table 2: Timing Analysis Report

Name	Slack ¹	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	∞	13	11	33	A11_imag[3]	X15_imag[15]	10.561	5.610	4.951
↳ Path 2	∞	13	11	33	A27_imag[3]	X23_imag[15]	10.561	5.610	4.951
↳ Path 3	∞	13	11	33	A27_imag[3]	X31_imag[15]	10.561	5.610	4.951
↳ Path 4	∞	13	11	33	A11_imag[3]	X7_imag[15]	10.561	5.610	4.951
↳ Path 5	∞	13	11	33	A11_imag[3]	X15_imag[13]	10.556	5.605	4.951
↳ Path 6	∞	13	11	33	A27_imag[3]	X23_imag[13]	10.556	5.605	4.951
↳ Path 7	∞	13	11	33	A27_imag[3]	X31_imag[13]	10.556	5.605	4.951
↳ Path 8	∞	13	11	33	A11_imag[3]	X7_imag[13]	10.556	5.605	4.951
↳ Path 9	∞	13	11	33	A11_imag[3]	X15_imag[14]	10.496	5.545	4.951
↳ Path 10	∞	13	11	33	A27_imag[3]	X23_imag[14]	10.496	5.545	4.951

4.1.4 Power Consumption

From the power analysis report shown in figure 7, we can observe that the total on-chip power consumption is approximately 1000.115 W, which is significantly increased. The junction temperature is also maximized to 125°C. There is no clock gating, and thus all modules are running all the time. This is a major drawback of the current design and confirms that clock gating is a necessary power optimization technique.

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	1000.115 W (Junction temp exceeded!)
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	125.0°C
Thermal Margin:	-1809.8°C (-966.8 W)
Effective SJA:	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low
Launch Power Constraint Advisor to find and fix invalid switching activity	

Figure 7: Power Analysis Report

4.2 Analysis of 32- Point FFT with Clock Gating

4.2.1 Simulation Waveform:

The simulation waveform of the proposed 32-point FFT architecture demonstrates the functionality of the design with clock gating enabled. The presence of the enable (en) signal controls the clock activity, allowing the circuit to operate only when required. Initially, undefined states (X) are observed, which later settle to valid values after proper signal propagation. Compared to the existing design, switching activity is reduced because the clock is gated when the enable signal is inactive. This confirms correct functional behavior along with improved power efficiency.

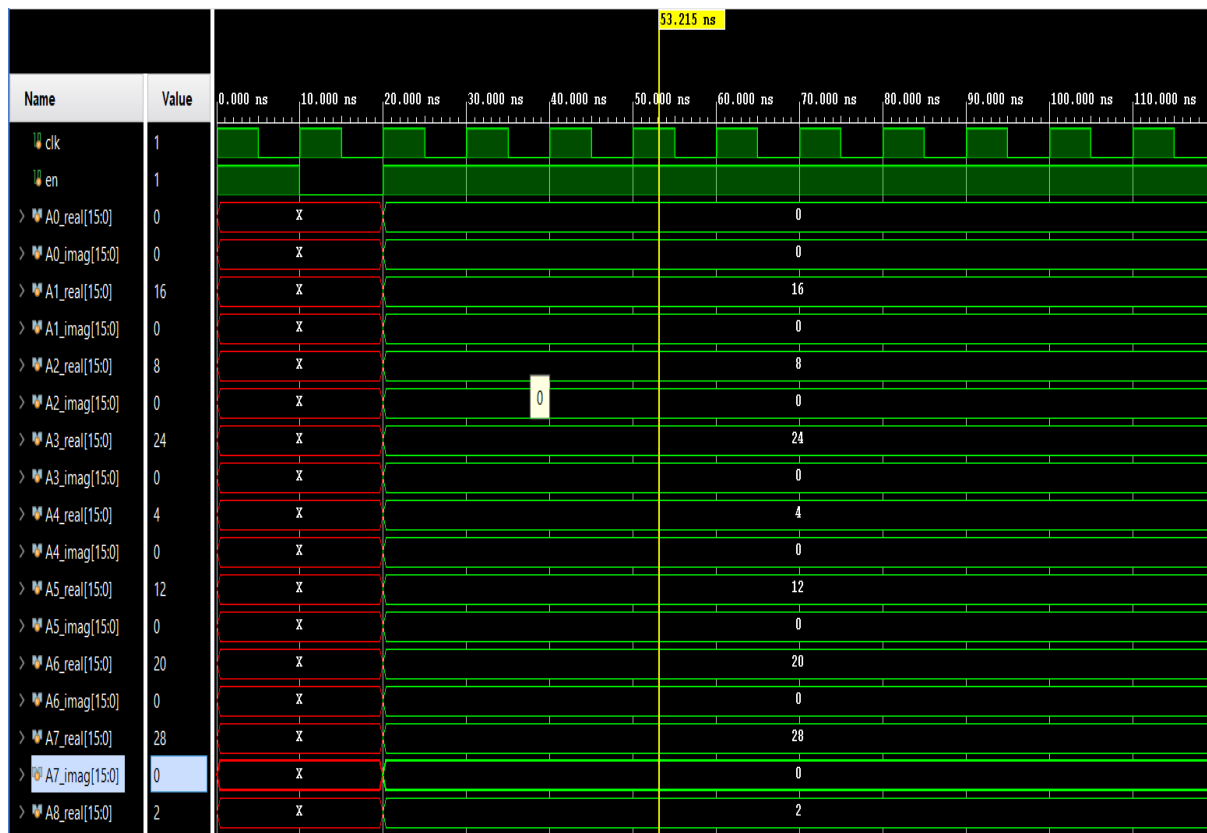


Figure 8: Simulation Waveform of 32-Point FFT with Clock Gating

4.2.2 Area Utilization

The area utilization report indicates that the proposed design uses approximately 3307 Slice LUTs and 897 Slice Registers. Compared to the existing design, there is a slight increase in LUT usage due to additional clock gating logic. However, this increase is minimal and acceptable considering the significant power savings achieved.

Name	^1	Slice LUTs (134600)	Slice Registers (269200)
▼ N top_32		3307	897
> I f1 (flipflop_based_clk)		0	1
I u1 (top_16)		1504	448
I u2 (top_16_0)		1504	448

Figure 9: Device Utilization

4.2.3 Delay Analysis

The delay analysis for the proposed design in table 3 shows that there is an improved critical path delay, which is around 7.406 ns, significantly less than that for the existing design, which is around ~10.561 ns. The logic delay is around 3.100 ns, while the net delay is around 4.306 ns. The delay has been reduced due to optimal switching activities and better management of signal propagation with clock gating techniques. The timing constraints are met with a positive slack.

Table 3: Timing Analysis Report

Name	Slack	^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
↳ Path 1	∞		12	9	33	A11_imag[3]	u1/X7_imag_reg[13]/D	7.406	3.100	4.306
↳ Path 2	∞		12	9	33	A27_imag[3]	u2/X7_imag_reg[13]/D	7.406	3.100	4.306
↳ Path 3	∞		12	9	33	A11_imag[3]	u1/X7_imag_reg[15]/D	7.404	3.098	4.306
↳ Path 4	∞		12	9	33	A27_imag[3]	u2/X7_imag_reg[15]/D	7.404	3.098	4.306
↳ Path 5	∞		12	9	33	A11_imag[3]	u1/X7_imag_reg[14]/D	7.343	3.037	4.306
↳ Path 6	∞		12	9	33	A27_imag[3]	u2/X7_imag_reg[14]/D	7.343	3.037	4.306
↳ Path 7	∞		12	9	33	A11_imag[3]	u1/X7_imag_reg[12]/D	7.320	3.014	4.306
↳ Path 8	∞		12	9	33	A27_imag[3]	u2/X7_imag_reg[12]/D	7.320	3.014	4.306
↳ Path 9	∞		11	9	33	A11_imag[3]	u1/X7_imag_reg[9]/D	7.308	3.002	4.306
↳ Path 10	∞		11	9	33	A27_imag[3]	u2/X7_imag_reg[9]/D	7.308	3.002	4.306

6.2.4 Power Consumption

From the power analysis report shown in figure 10, it is observed that the total on-chip power consumption is minimized to approximately 62.447 W. This is a significant improvement when compared to the existing design, which consumes approximately 1000 W. Although the junction temperature is maintained at 125°C, the total power consumption is significantly minimized due to a reduction in switching activity. Clock gating is effective in disabling

modules that are not in use.

Total On-Chip Power:	62.447 W (Junction temp exceeded!)
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	125.0°C
Thermal Margin:	-56.7°C (-29.1 W)
Effective θ_{JA} :	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Figure 10: Power Analysis Report

4.3 Comparative Analysis of 32-Point FFT with and without Clock Gating

The comparison between the existing 32-point FFT architecture without clock gating and the proposed architecture incorporating clock gating is carried out based on the essential parameters like area, power consumption, and delay, and the results are presented in Table 4. From the results, it is clear that the area utilization increases slightly from 3290 LUTs to 3307 LUTs in the proposed architecture, which is due to the introduction of clock gating logic in the design. A significant improvement is noted in the power consumption aspect, as the power consumption is reduced significantly from 1000.115 W in the existing architecture to 62.447 W in the proposed architecture.

As far as the delay is concerned, the proposed architecture shows significant improvement in the delay aspect, as the delay is reduced significantly from 10.561 ns in the existing architecture to 7.406 ns in the proposed architecture. From the results, it is clear that the 32-point FFT architecture incorporating clock gating is a power-efficient and faster architecture compared to the existing architecture, with a slight increase in the area utilization aspect. From the results, it is clear that clock gating is an effective technique in the optimization of digital architectures, and the technique is useful in the power-constrained applications of digital systems.

Table 4: Comparative Analysis of 32-Point FFT with and without Clock Gating

S.No	Method	Area (LUT)	Power(W)	Delay(ns)
1	Without Clock	3290	1000.115	10.561
2	With Clock	3307	62.447	7.406

5. Conclusion

Fast Fourier Transform (FFT) is considered to be at the core of digital signal processing and communication systems. However, it is also considered to have high resource requirements. In this article, the authors have proposed a novel low-power and low-complexity architecture of the FFT algorithm that makes use of an innovative input grouping method to minimize the number of multiplications between the inputs and the twiddle factors of the FFT. In addition to this, a partial sum sharing method is also presented to minimize hardware resource

requirements. Logic synthesis results of the ASIC design of the proposed 32-point FFT architecture have been presented to validate the efficiency of the proposed method.

References

- [1] W. Hong et al., “Multibeam antenna technologies for 5G wireless communications,” *IEEE Trans. Antennas Propag.*, vol. 65, no. 12, pp. 6231–6249, Dec. 2017.
- [2] P. K. Daram and C.-I. H. Chen, “Smart FFT measurement for reconfigurable sensor using a wideband digital receiver,” in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, Aug. 2021, pp. 200–203.
- [3] J. Chen, J. Hu, S. Lee, and G. E. Sobelman, “Hardware efficient mixed radix-25/16/9 FFT for LTE systems,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 2, pp. 221–229, Feb. 2015.
- [4] N. Lei and X. Gu, “FFT-OT: A fast algorithm for optimal transportation,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 6260–6269.
- [5] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Math. Comput.*, vol. 19, pp. 297–301, Apr. 1965.
- [6] J. Wang, S. Li, and X. Li, “Scheduling of data access for the radix-2k FFT processor using single-port memory,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1676–1689, Jul. 2020.
- [7] J. Hazarika, M. T. Khan, and S. R. Ahamed, “Low-complexity continuous-flow memory-based FFT architectures for real-valued signals,” in *Proc. 32nd Int. Conf. VLSI Des. and 18th Int. Conf. Embedded Syst. (VLSID)*, Jan. 2019, pp. 46–51.
- [8] S. Liu and D. Liu, “A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 3, pp. 511–523, Mar. 2019.
- [9] T. Kawabata and H. Tsutsui, “An evaluation of high-throughput scalable radix-4 FFT processor architecture using fixed-point arithmetic,” in *Proc. Asia-Pacific Signal Inf. Process. Assoc. Annu. Summit Conf. (APSIPA ASC)*, Dec. 2020, pp. 114–117.
- [10] N. Nadar, M. Mehta, K. Bhat, J. Mendes, and R. Chaudhari, “Hardware implementation of pipelined FFT using polyphase decomposition,” in *Proc. Int. Conf. Recent Trends Electron. Inf. Commun. Technol.*, Aug. 2021, pp. 339–343.
- [11] M. Parfieniuk, “Lifting-based alternatives to the FFT for computing the 4-, 8-, and 16-point discrete Fourier transforms,” in *Proc. 29th Eur. Signal Process. Conf. (EUSIPCO)*, Aug. 2021, pp. 2184–2188.
- [12] IEEE Standard for Information Technology—Telecommunications and Information Exchange Between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical

Layer (PHY) Specifications Amendment 3: Enhancements for Very High Throughput in the 60 GHz Band, IEEE Standard 802.11ad-2012, Dec. 2012.

- [13]B. Liu et al., “Precision adaptive MFCC based on R2SDF-FFT and approximate computing for low-power speech keywords recognition,” *IEEE Circuits Syst. Mag.*, vol. 21, no. 4, pp. 24–39, 4th Quart., 2021.
- [14]H.-Y. Chin, P.-Y. Tsai, and S.-Y. Lee, “Implementation of a two-dimensional FFT/IFFT processor for real-time high-resolution synthetic aperture radar imaging,” in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2021, pp. 211–216.
- [15]S.-C. Hsu, S.-J. Huang, S.-G. Chen, S.-C. Lin, and M. Garrido, “A 128-point multi-path SC FFT architecture,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Oct. 2020, pp. 1–5.
- [16]M. Garrido and P. Paz, “Optimum MDC FFT hardware architectures in terms of delays and multiplexers,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 3, pp. 1003–1007, Mar. 2021.
- [17]X. Han, J. Chen, and S. Rahardja, “A new twiddle factor merging method for low complexity and high speed FFT architecture,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Sep. 2019, pp. 1–5.
- [18]X. Han, J. Chen, B. Qin, and S. Rahardja, “A novel area-power efficient design for approximated small-point FFT architecture,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4816–4827, Dec. 2020.
- [19]T.-W. Siu, C.-W. Sham, and F. C. M. Lau, “Operating frequency improvement on FPGA implementation of a pipeline large-FFT processor,” in *Proc. 19th Int. Conf. Adv. Commun. Technol. (ICACT)*, Feb. 2017, pp. 5–9.
- [20]T.-T. Hoang, D.-H. Le, and C.-K. Pham, “VLSI design of floating-point twiddle factor using adaptive CORDIC on various iteration limitations,” in *Proc. IEEE 12th Int. Symp. Embedded Multicore/Many-Core Syst. Chip*, Sep. 2018, pp. 225–232.
- [21]W. M. Gentleman and G. Sande, “Fast Fourier transforms: For fun and profit,” in *Proc. AFIPS Fall Joint Comput. Conf.*, San Francisco, CA, USA, Nov. 1966, pp. 563–578.
- [22]J. Ding, J. Chen, and C.-H. Chang, “A new paradigm of common subexpression elimination by unification of addition and subtraction,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1605–1617, Oct. 2016.
- [23]M. Garrido and P. Malagón, “The constant multiplier FFT,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 322–335, Jan. 2021.

